
コンピュータビジョン特論A/B
現代脳計算論

Computer Vision A/B
Modern Neural Computation

Keisuke Yoneda, Kanazawa University
Email: k.yoneda@staff.kanazawa-u.ac.jp

Supervised Learning

116

❑ Machine Learning

- Supervised Learning

❑ Perceptron

- Multilayer Perceptron
- Backpropagation

❑ Supervised Learning

- Support Vector Machine
- AdaBoost
- Random Forest

Machine Learning?

117

□ Training a Task

- E.g. Classification Task: Distinguish the category of input data

□ Supervised Learning

- Given: Set of input (x) and output (y)
- Train function f ($y = f(x)$)

□ Unsupervised Learning

- Given: input (x)
- Learn relationship between input data

□ Reinforcement Learning

- Given: reward
- Train policy function f ($y=f(x)$) maximizing reward

Simple Perceptron

118

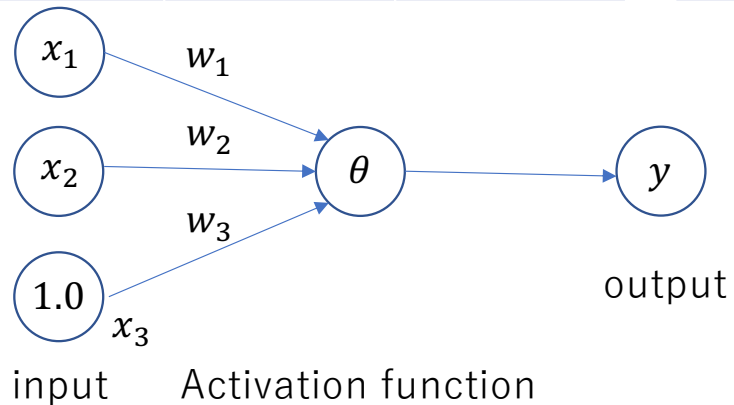
Example: logical operation

x_1	x_2	$y(AND)$
0	0	0
0	1	0
1	0	0
1	1	1

w_1	w_2	w_3
1.0	1.0	-1.5

x_1	x_2	$y(OR)$
0	0	0
0	1	1
1	0	1
1	1	1

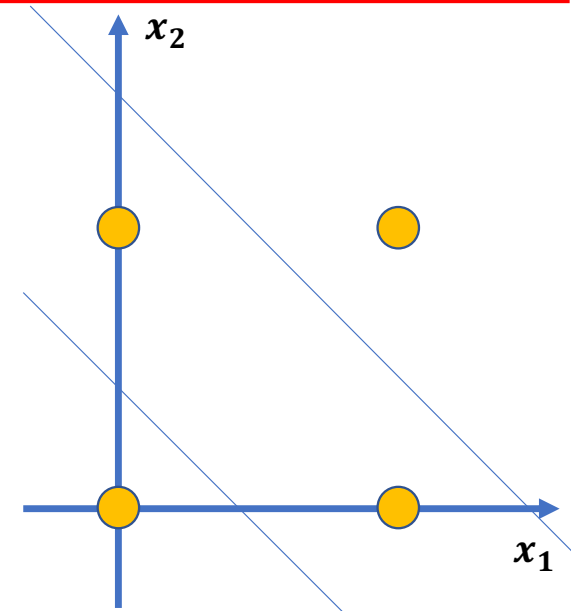
w_1	w_2	w_3
1.0	1.0	-0.5



$$\theta(u) = \begin{cases} 1 & (u \geq 0) \\ 0 & (\text{otherwise}) \end{cases}$$

$$u = \sum x_i w_i$$

$$y = \theta(u)$$

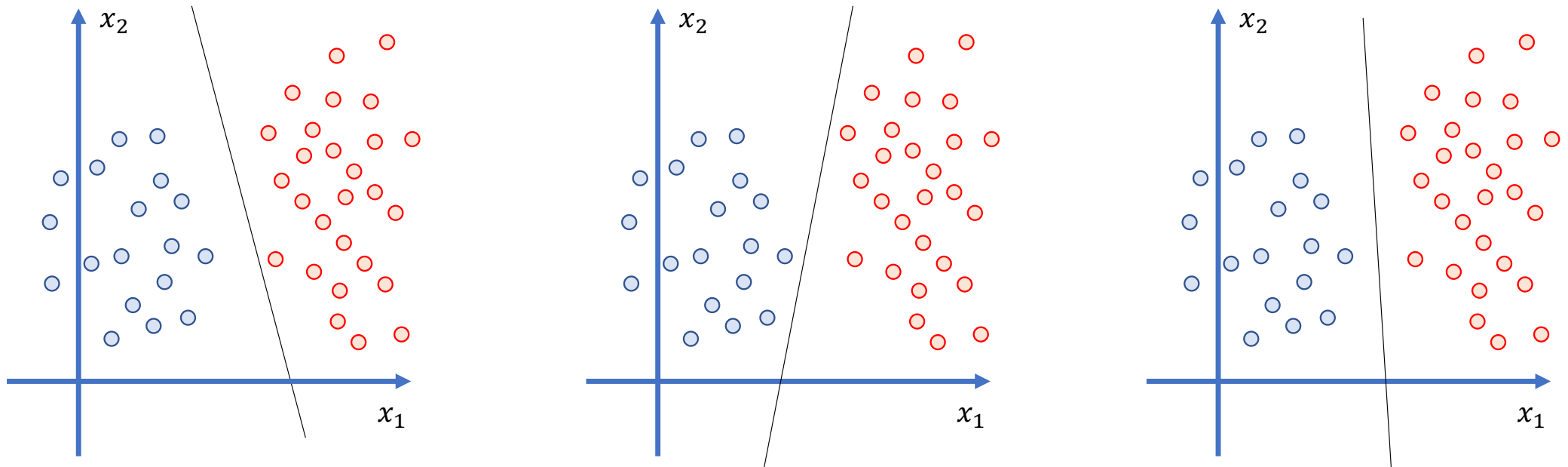


$$(AND): u = x_1 + x_2 - 1.5 = 0 \\ \therefore x_2 = -x_1 + 1.5$$

$$(OR): u = x_1 + x_2 - 0.5 = 0 \\ \therefore x_2 = -x_1 + 0.5$$

Support Vector Machine (1/2)

119



□ How to separate data?

Support Vector Machine (2/2)

120

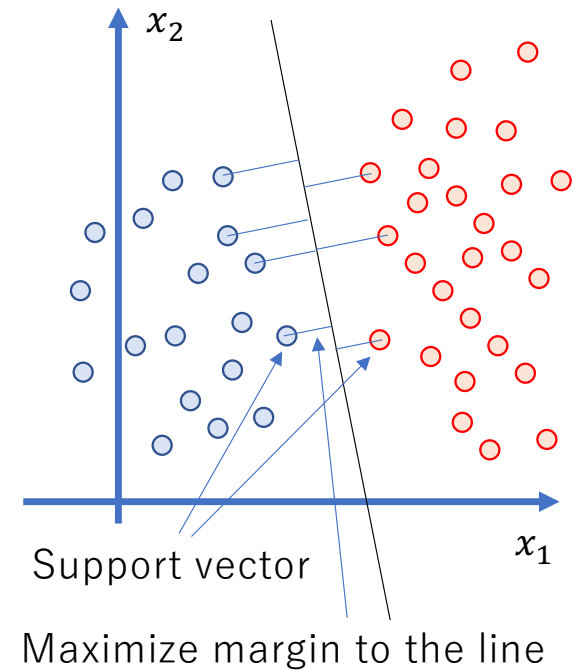
Machine learning method for Supervised Learning

- Linear SVM (Support Vector Machine)

- ◆ 2-class classification method
- ◆ Optimize a hyperplane as a linear function
 - Maximize margin distance

- Non-Linear SVM

- ◆ 2-class classification method
- ◆ Add a kernel function based on non-linear function
- ◆ Optimize a hyperplane for more complicated data



Linear SVM (1/5)

121

□ Linear SVM

- Optimize a hyperplane as a linear function

- ◆ (In case of 2-D), find a line $w_1x_1 + w_2x_2 + b = 0$

- $w^T x + b = 0, (w = [w_1, w_2]^T, x = [x_1, x_2]^T)$

- ◆ Distance to the line from a point x_i^+

- $\frac{|w^T x_i^+ + b|}{||w||}$

- Separate data (Class+, Class-)

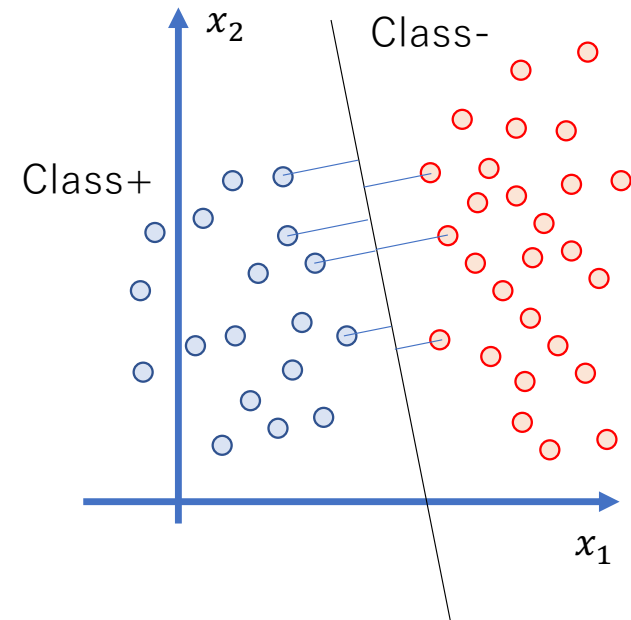
- ◆ Class+: $w^T x_i^+ + b > 0$

- ◆ Class-: $w^T x_i^- + b < 0$

□ Maximize margin

- maximize $M = \frac{(w^T x^+ + b)}{||w||} = \frac{-(w^T x^- + b)}{||w||}, (x^+, x^-: \text{support vector})$

- ◆ subject to $\frac{y_i(w^T x_i + b)}{||w||} \geq M, y_i = 1(\text{class } +), \text{ or } -1(\text{class } -)$



Linear SVM (2/5)

Linear SVM = Simple Perceptron??

122

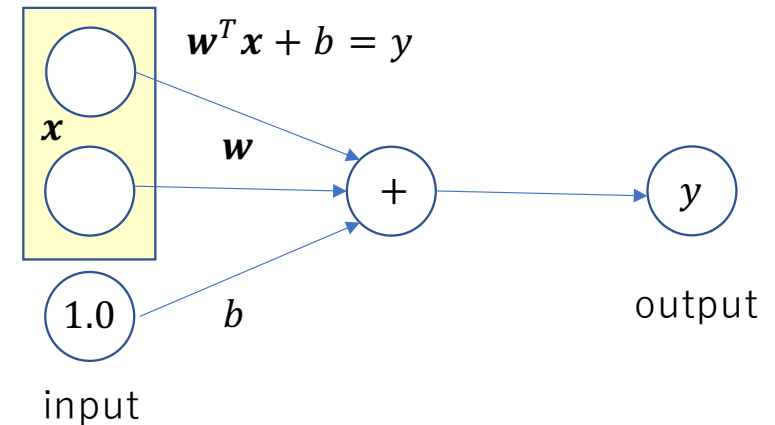
□ Maximize margin

- maximize M , subject to $\frac{y_i(\mathbf{w}^T \mathbf{x}_i + b)}{\|\mathbf{w}\|} \geq M$
 - ◆ $y_i = 1$ (class +), or -1 (class -)

normalization

$$M = \frac{1}{\|\mathbf{w}\|} \quad \frac{y_i(\mathbf{w}^T \mathbf{x}_i + b)}{\|\mathbf{w}\|} \geq \frac{1}{\|\mathbf{w}\|} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$$

$\mathbf{w}$: directional vector



□ Convert Maximization to Minimization

- maximize $\frac{1}{\|\mathbf{w}\|}$, subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$
- minimize $f(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2$, subject to $g_i(\mathbf{w}, b) = y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$

○ Linear SVM (3/5)

123

□ Support Vector Machine: Maximize Margin

- minimize $f(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2$, subject to $g_i(\mathbf{w}, b) = y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$

□ Lagrange Multiplier (ラグランジュの未定乗数法)

- Convert conditional extremum problem to unconditional extremum problem
- maximize $f(x)$, subject to $g_i(x) \leq 0$
 - ◆ $f(x), g_i(x)$: continuously differentiable
 - ◆ $f(a)$ is extremum (極値)
- Lagrange function $L(x, \lambda) = f(x) - \sum_i \lambda_i g_i(x)$
 - ◆ If $f(a)$ is extremum (極値),
 - Then, $\frac{\partial L}{\partial x}(a, \lambda') = \frac{\partial L}{\partial \lambda}(a, \lambda') = 0$
 - $\lambda_i \geq 0, \lambda_i g_i(x) = 0$

Linear SVM (4/5)

124

□ Support Vector Machine: Maximize Margin

- minimize $f(\mathbf{w}) = \frac{1}{2} \|\mathbf{w}\|^2$, subject to $g_i(\mathbf{w}, b) = y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$

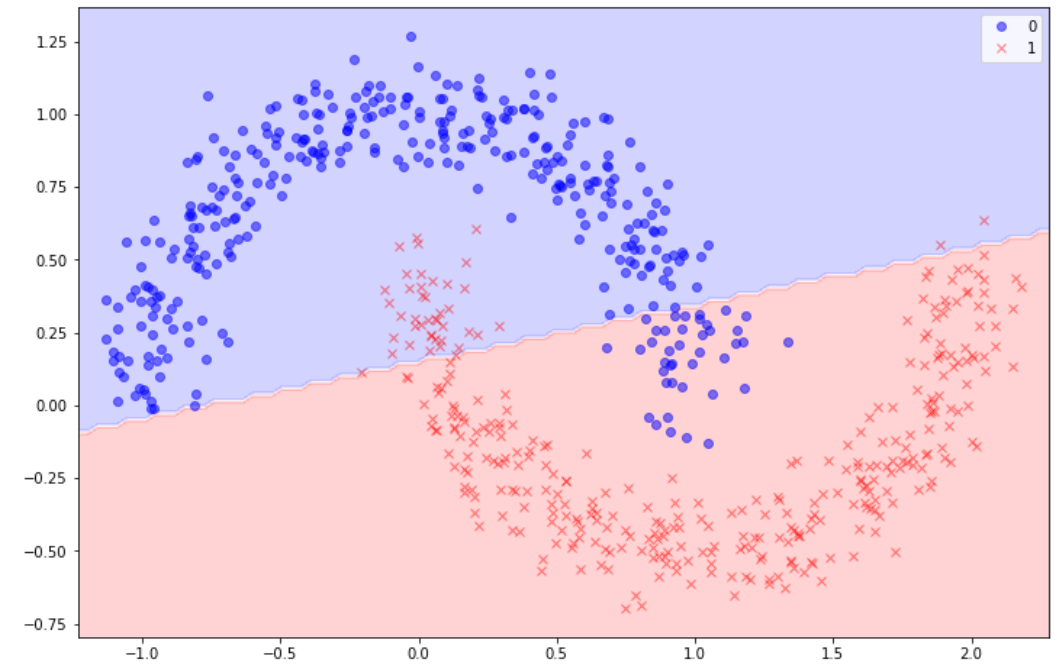
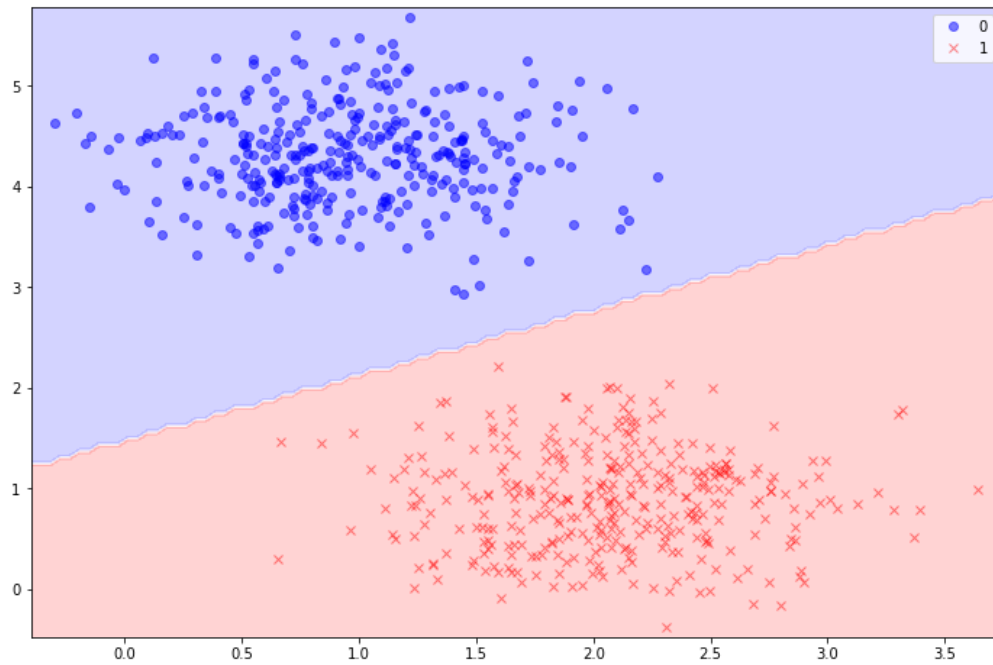
$$\square L(\mathbf{w}, b, \lambda) = f(\mathbf{w}) - \sum_i \lambda_i g_i(\mathbf{w}, b) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \lambda_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1)$$

- $\frac{\partial L}{\partial \mathbf{w}} = 0$, then, $\mathbf{w} = \sum_i \lambda_i y_i \mathbf{x}_i$
- $\frac{\partial L}{\partial b} = 0$, then, $\sum_i \lambda_i y_i = 0$
- $\lambda_i \geq 0, y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0, \lambda_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1) = 0$
- $L(\mathbf{w}, b, \lambda) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \lambda_i y_i \mathbf{w}^T \mathbf{x}_i - \sum_i \lambda_i y_i b + \sum_i \lambda_i$
- $= \frac{1}{2} \|\mathbf{w}\|^2 - \mathbf{w}^T \sum_i \lambda_i y_i \mathbf{x}_i - b \sum_i \lambda_i y_i + \sum_i \lambda_i = \frac{1}{2} \|\mathbf{w}\|^2 - \mathbf{w}^T \mathbf{w} + \sum_i \lambda_i$
- $= \frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \mathbf{w}^T \mathbf{w} + \sum_i \lambda_i$
- $\therefore \underset{\lambda}{\text{maximize}} L(\lambda) = \sum_i \lambda_i - \frac{1}{2} \sum_i \sum_j \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$
 - ◆ $\lambda_i \geq 0, \sum_i \lambda_i y_i = 0$

Quadratic Programming, 二次計画問題

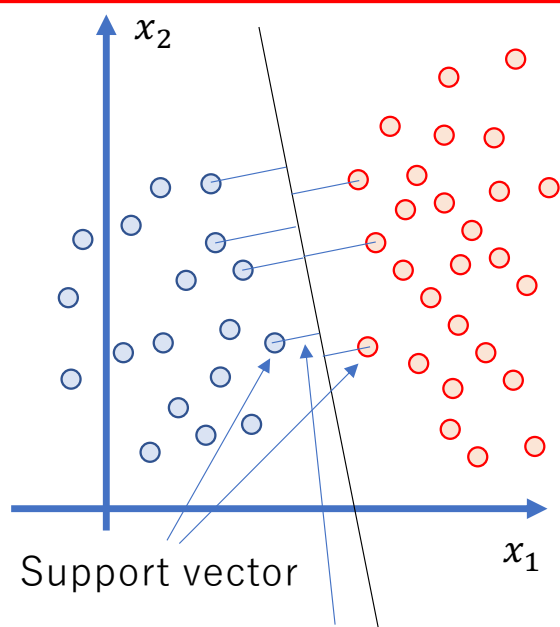
Linear SVM (5/5)

125

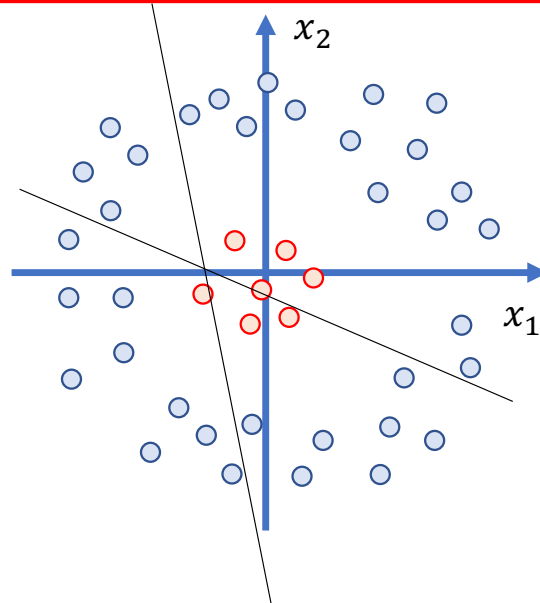


Non-Linear SVM (1/3)

126



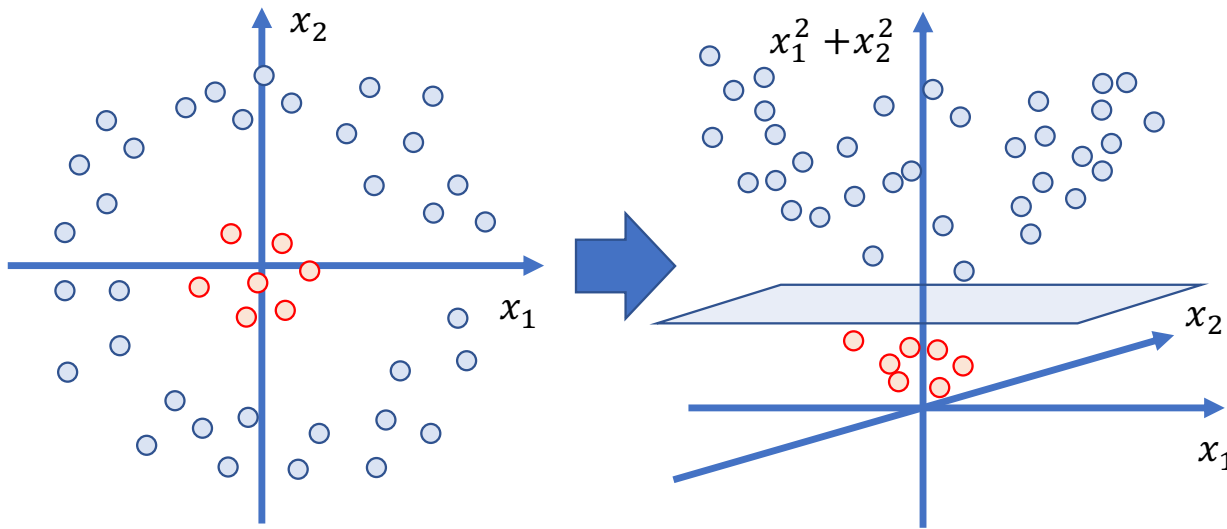
Maximize margin to the line



How to separate data?

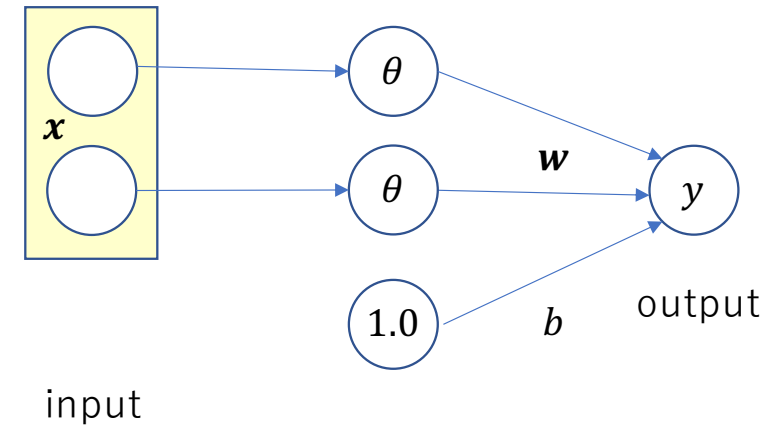
Non-Linear SVM (2/3)

127



Add additional feature (e.q.: $x_1^2 + x_2^2$)

Non-Linear SVM = Multilayer Perceptron with non-linear activate function??

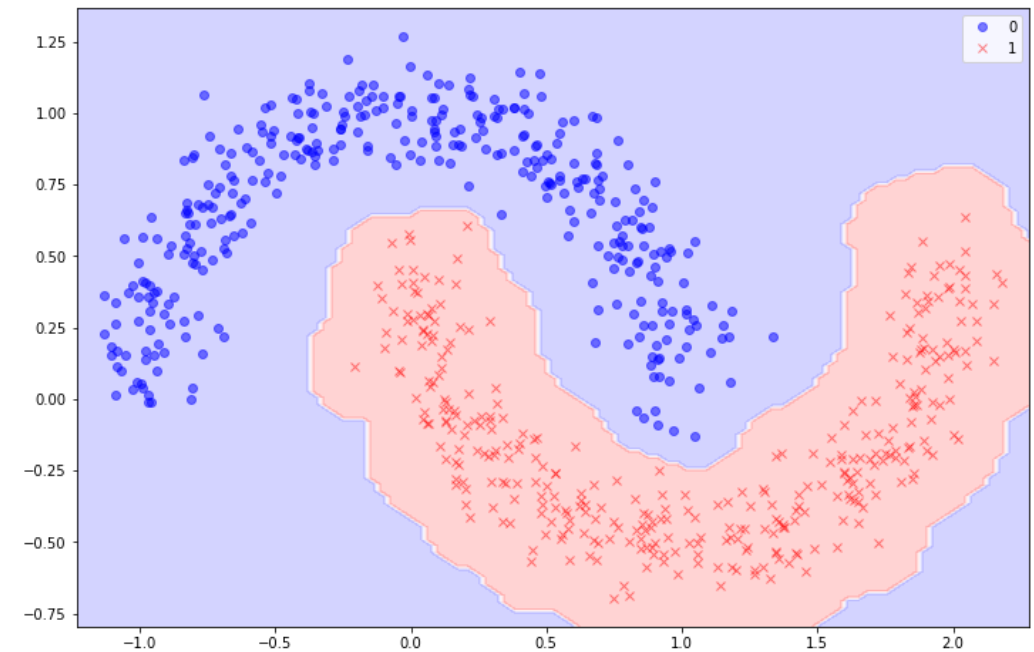
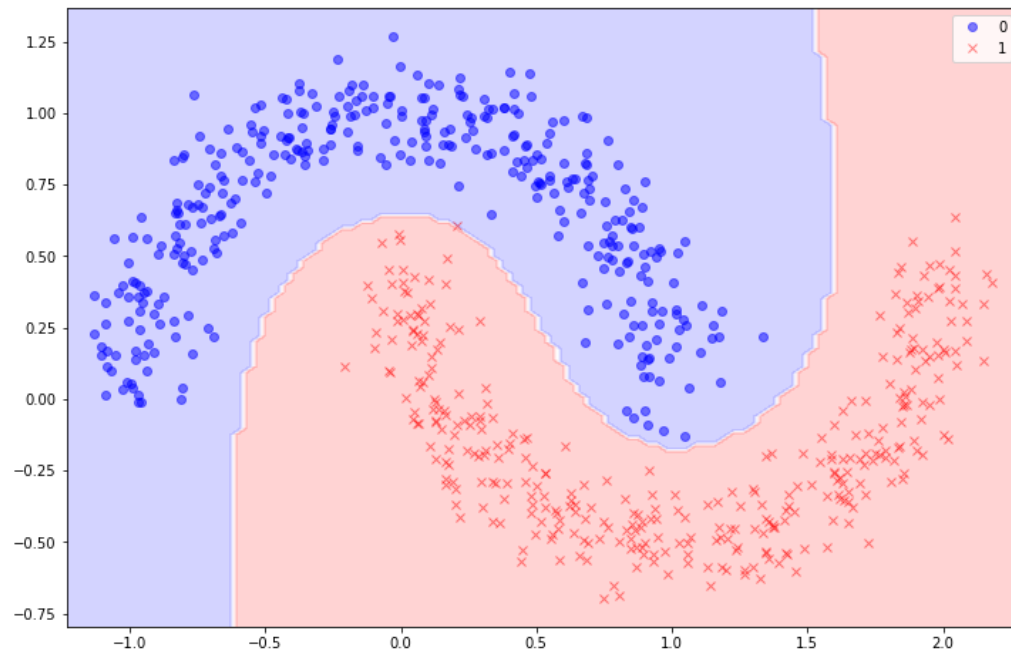


□ Kernel trick (Kernel method)

- Using non-linear function to mapping the data to additional feature space
- Basic kernel functions
 - ◆ Polynomial kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d$
 - ◆ Gauss (Radial Basis Function) kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2})$
 - ◆ Sigmoid Kernel: $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(b\mathbf{x}_i^T \mathbf{x}_j + c)$

Non-Linear SVM (3/3)

128



Boosting/Bagging (1/2)

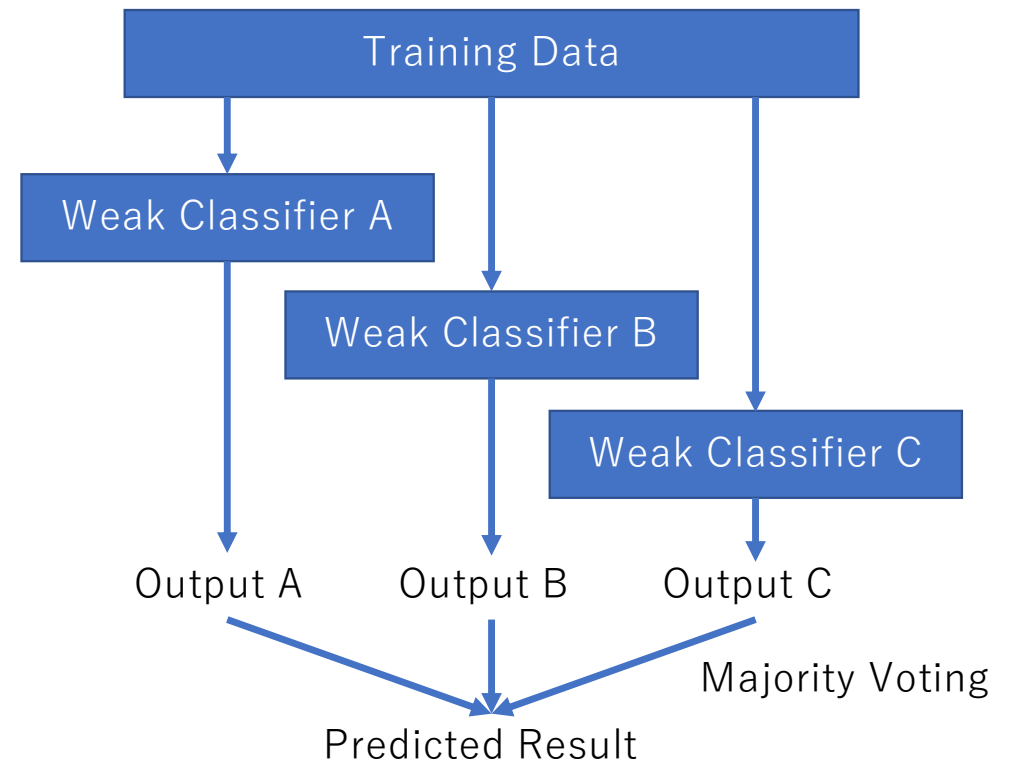
129

□ Boosting/Bagging

- Combine multiple weak classifier

□ Bagging

- Bootstrapping
 - ◆ Randomly select training data as sub data to generate weak classifier
- Predicted result is generated by majority vote of weak classifiers
- **Algorithm**
 - ◆ **Random Forest**



Boosting/Bagging (2/2)

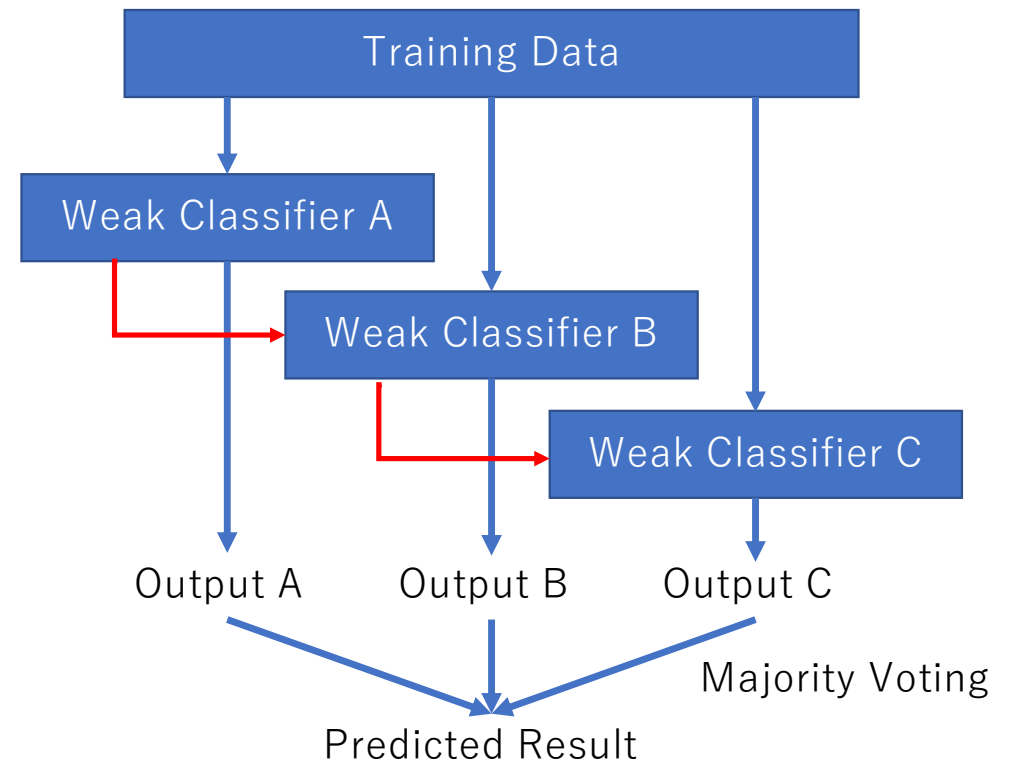
130

□ Boosting/Bagging

- Combine multiple weak classifier

□ Boosting

- Weighting weak classifiers based on error rate
 - ◆ The new weak classifier is designed to compensate for the disadvantage of previous classifiers
- Predicted result is generated by majority vote of weak classifiers
- Algorithm
 - ◆ AdaBoost

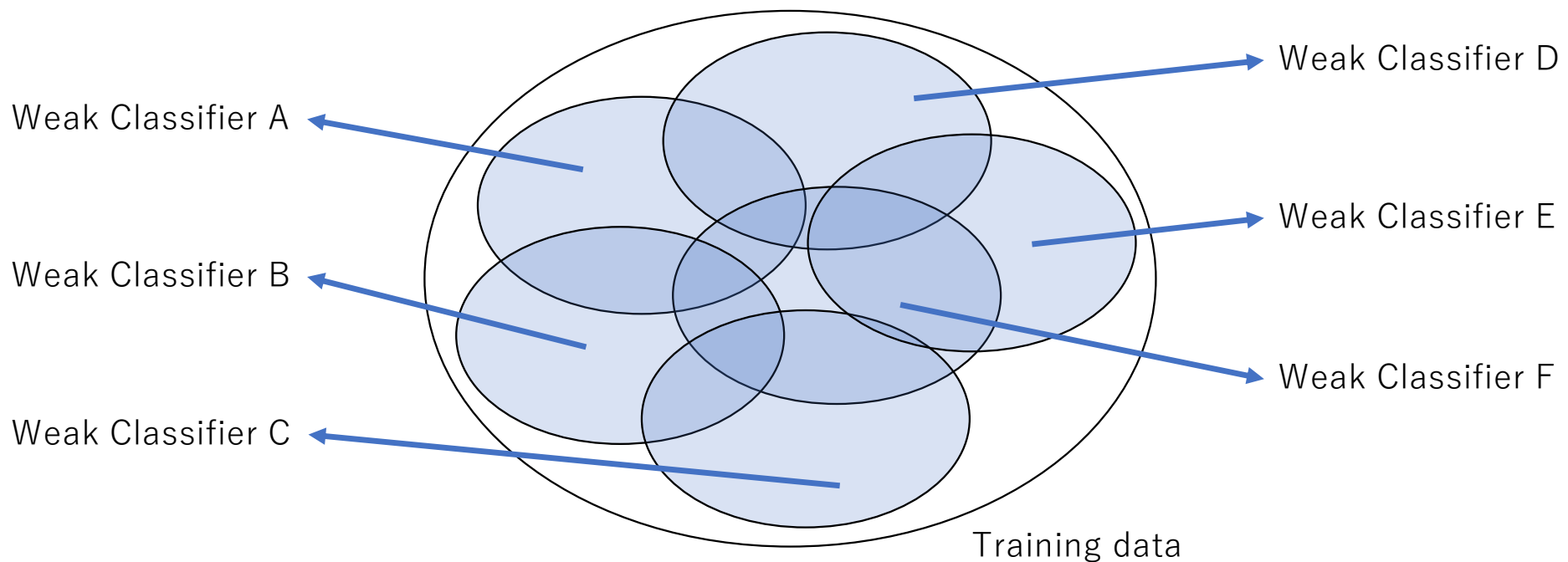


Random Forest

131

□ Bagging

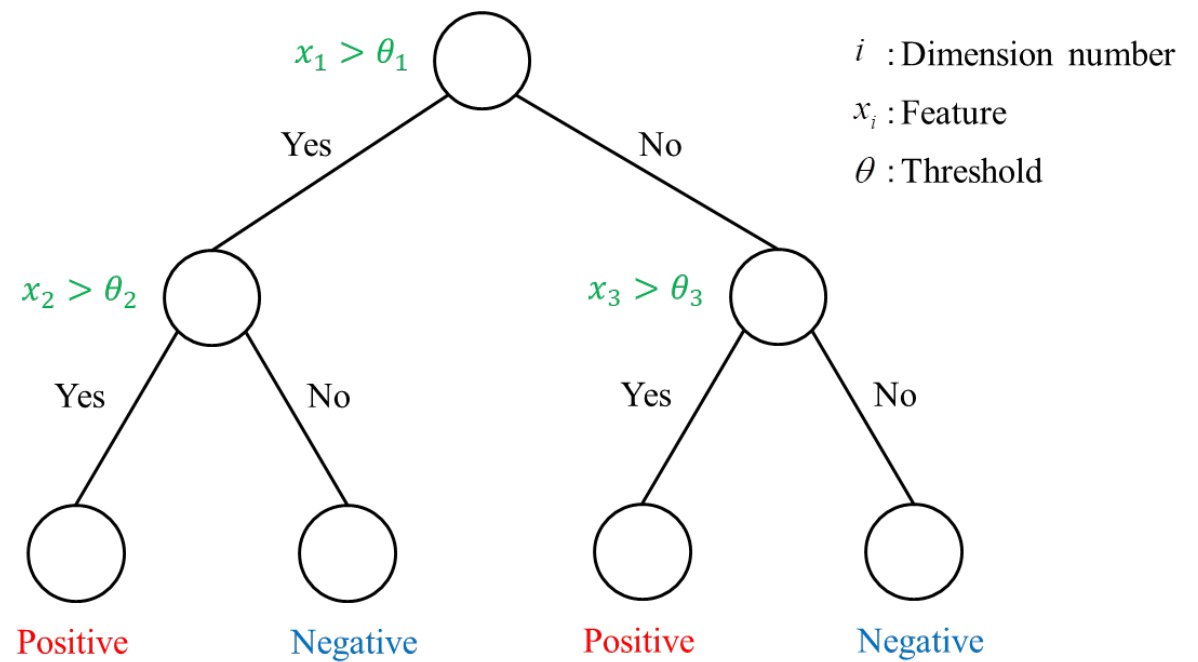
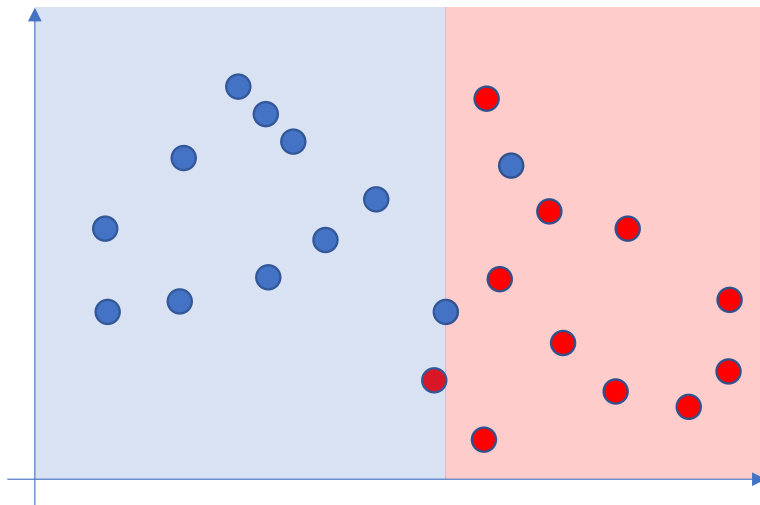
- Bootstrapping
 - ◆ Randomly select training data as sub data to generate weak classifier
- Predicted result is generated by majority vote of weak classifiers



Weak Classifier (1/2)

132

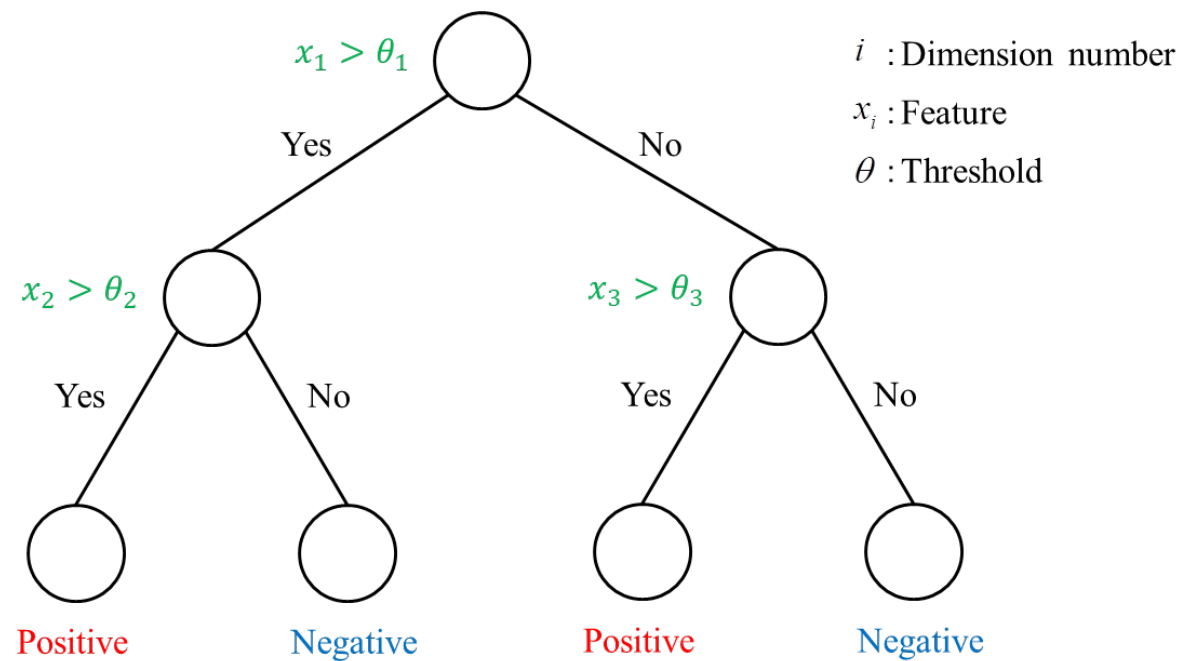
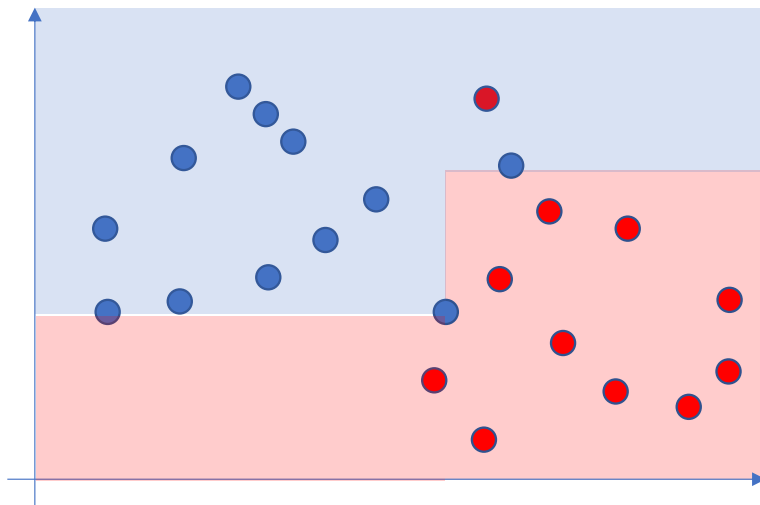
Decision Tree for weak classifier



Weak Classifier (2/2)

133

Decision Tree for weak classifier

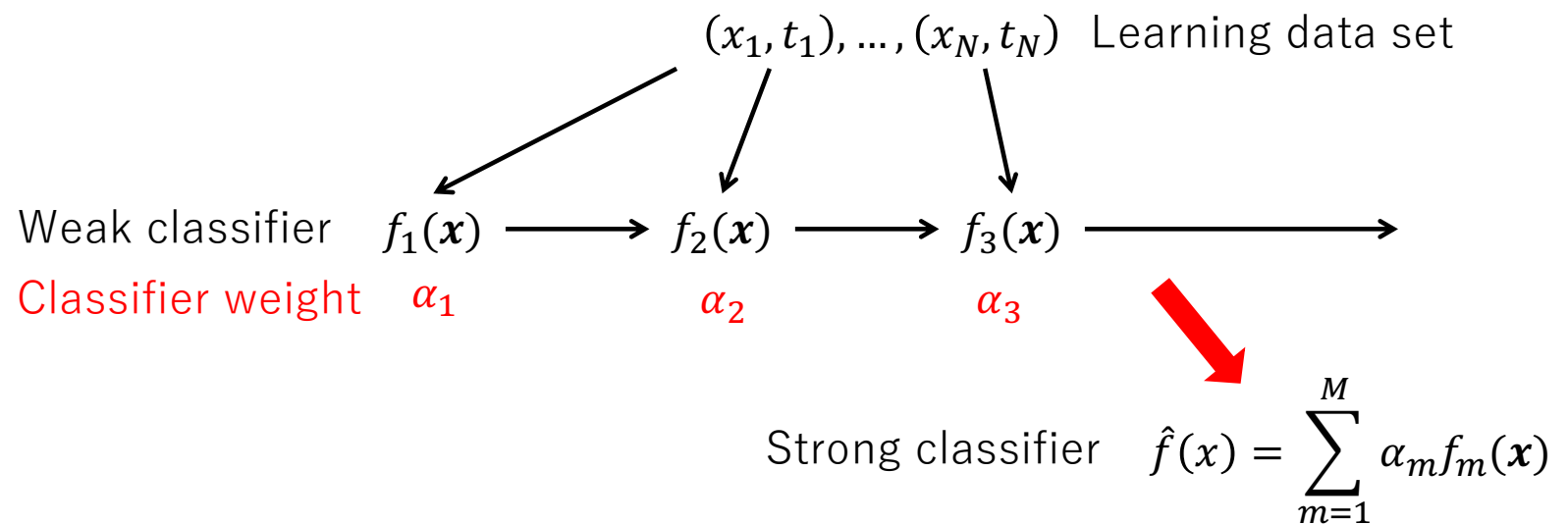


Boosting

134

□ Boosting

- Combine multiple weak classifier
 - ◆ Weighting each classifier based on error rate
- Output decision (e.q.: class -1 or class +1) by majority of weak classifiers



AdaBoost (1/2)

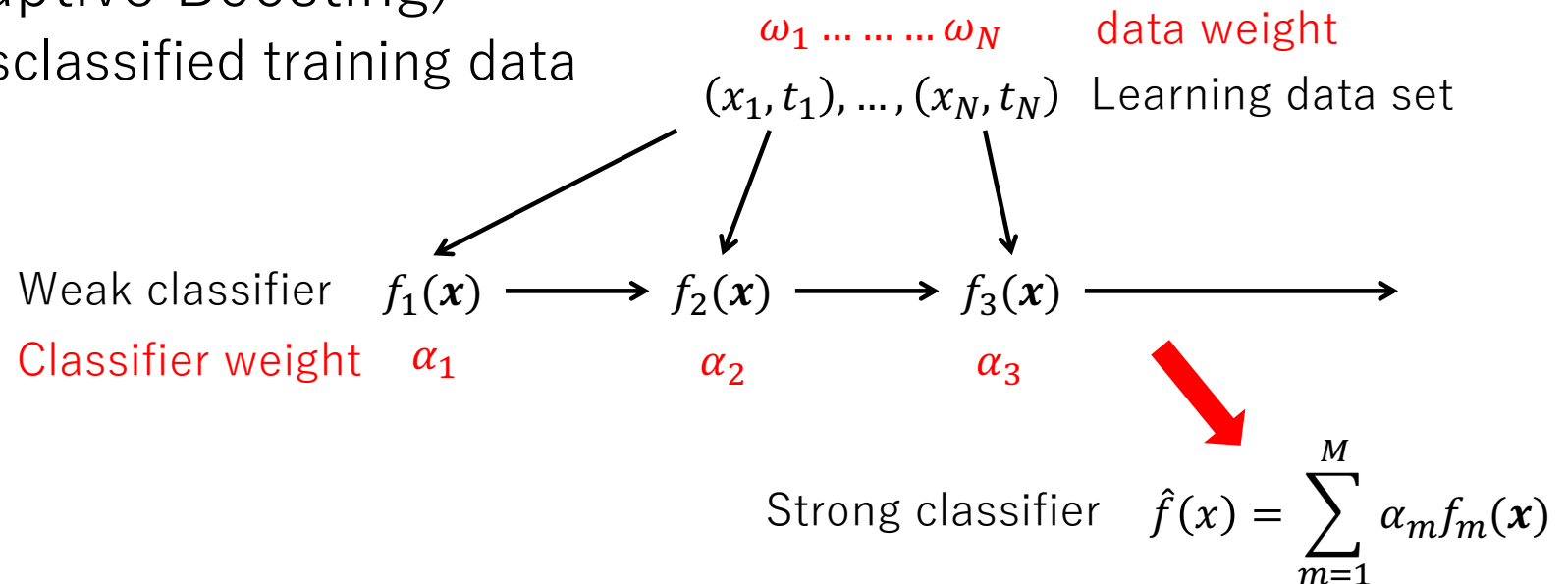
135

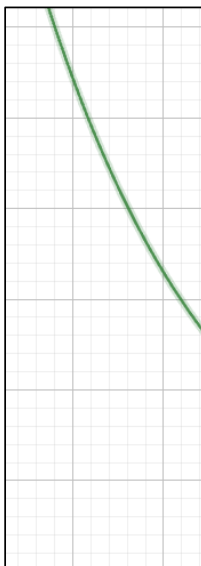
□ Boosting

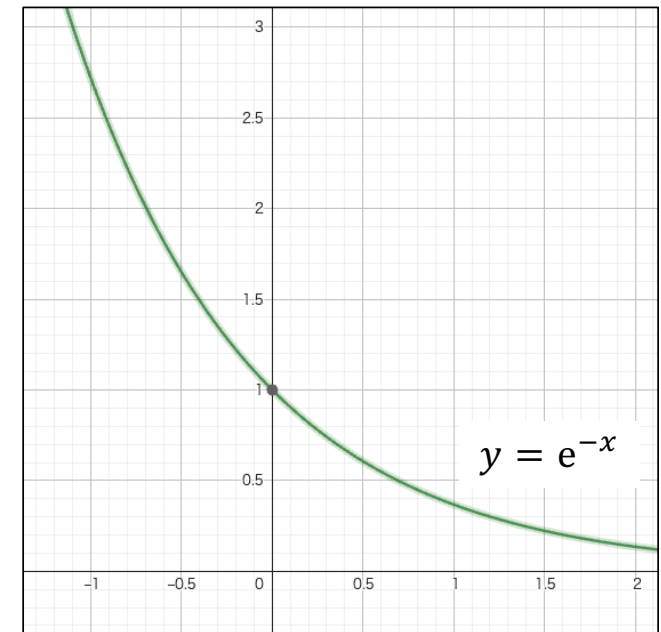
- Combine multiple weak classifier
 - ◆ Weighting each classifier based on error rate
- Output decision (e.q.: class -1 or class +1) by majority of weak classifiers

□ AdaBoost (Adaptive Boosting)

- Weighting misclassified training data



1. Prepare training dataset $(x_1, y_1), \dots, (x_N, y_N)$
 - $y_i = \{-1, +1\}$: label
 2. Initialize data weights with $\omega_i = \frac{1}{N}$
 3. Training step: For $m = 1, \dots, M$ (M : number of weak classifiers)
 - i. Determine new weak classifier h_m with minimum weighted error
 - ◆ Weighted error: $\sum_{i=1}^N \omega_i I(h_m(x_i) \neq y_i)$
 - ◆ $I(h_m(x_i) \neq y_i) = 1$ (if $h_m(x_i) \neq y_i$) or 0 (if $h_m(x_i) = y_i$)
 - ii. Normalize average error rate (if $\varepsilon_m < 0.5$, finish the training)
 - ◆ $\varepsilon_m = \frac{\sum_{i=1}^N \omega_i I(h_m(x_i) \neq y_i)}{\sum_{i=1}^N \omega_i}$
 - iii. Compute weight of weak classifier
 - ◆ $\alpha_m = \log\left(\frac{1-\varepsilon_m}{\varepsilon_m}\right)$ ε_m : small $\rightarrow$ large classifier weight
 ε_m : large $\rightarrow$ small classifier weight
 - iv. Update data weight
 - ◆ $\omega_i = \omega_i \exp\{-\alpha_m h_m(x_i) y_i\}$ True: decrease data weight
 False: increase data weight
 4. Output classification result
 - $H(x) = \text{sign}\{\sum_{m=1}^M \alpha_m h_m(x)\}$
- 



Machine Learning?

137

□ Training a Task

- E.g. Classification Task: Distinguish the category of input data

□ Supervised Learning

- Given: Set of input (x) and output (y)
- Train function f ($y = f(x)$)

□ Unsupervised Learning

- Given: input (x)
- Learn relationship between input data

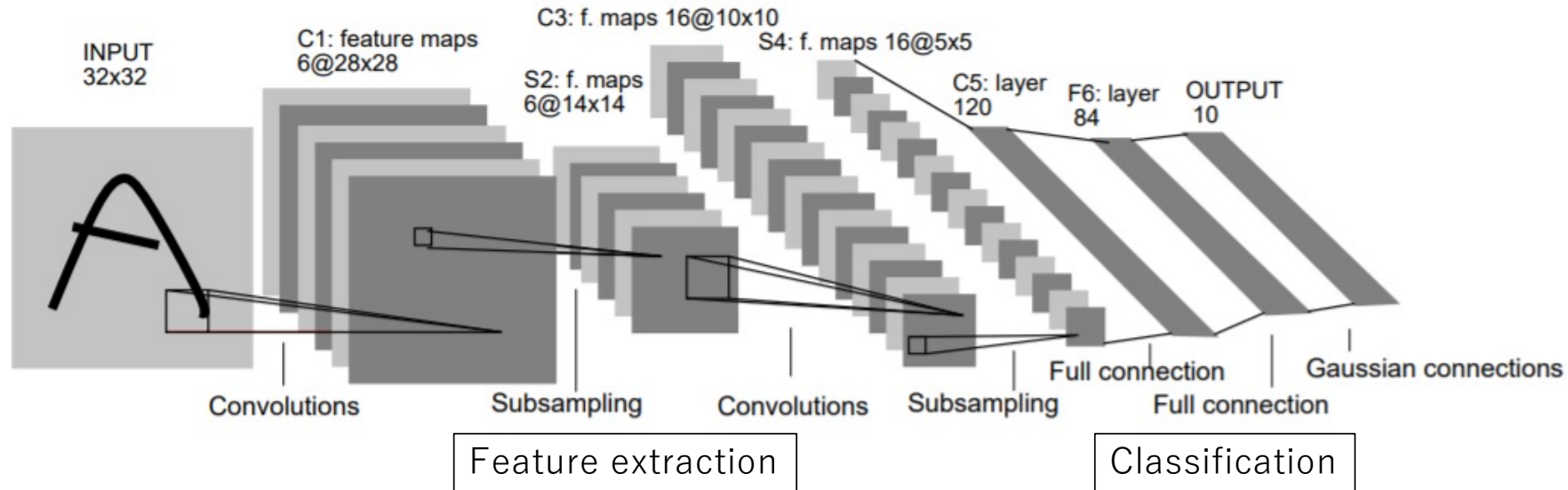
□ Reinforcement Learning

- Given: reward
- Train policy function f ($y=f(x)$) maximizing reward

Convolutional Neural Network

138

- ❑ Multi Layer Perceptron (MLP)
 - Input: array data
- ❑ Convolutional Neural Network (CNN)
 - Input: 2D image data ($W \times H \times C$)
 - ◆ Possible to handle 2D spatial information

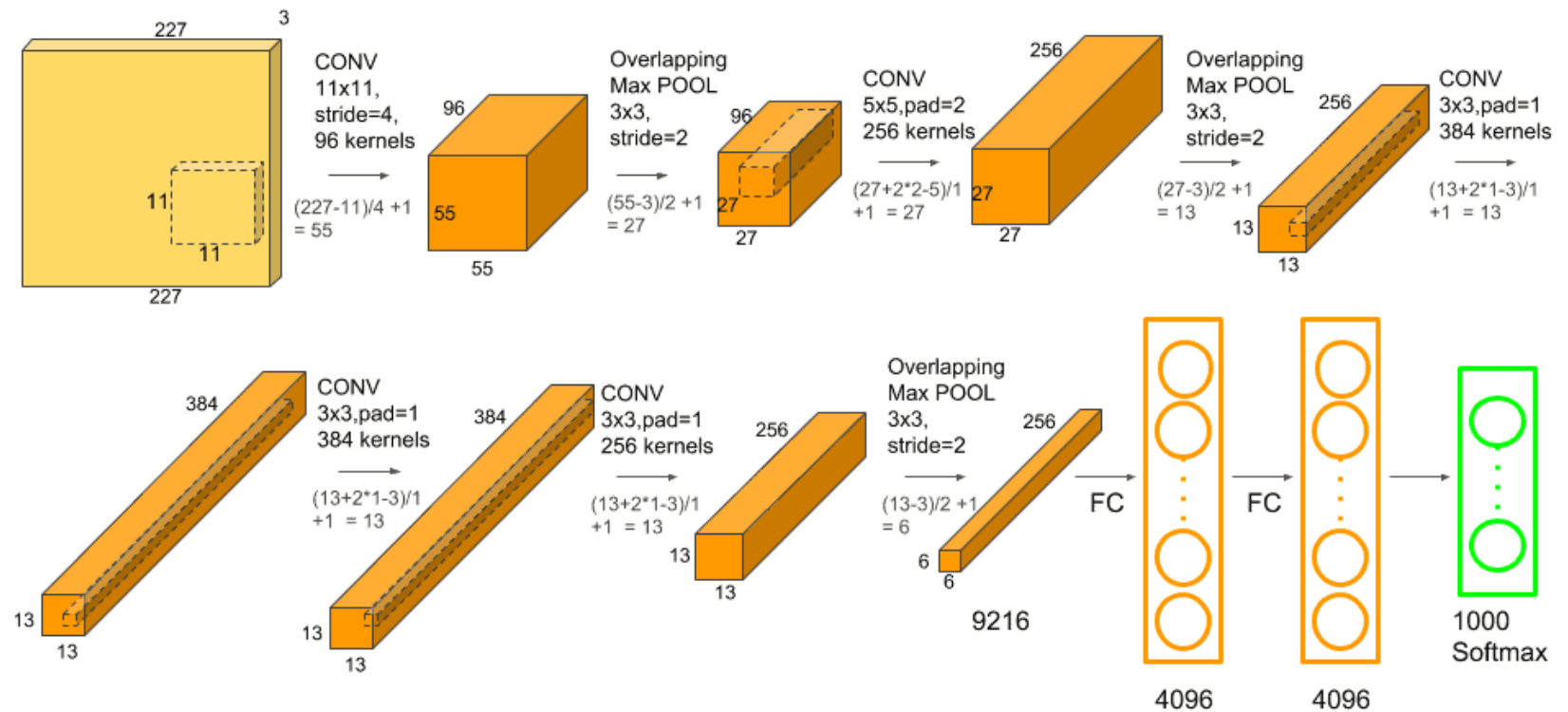


LeNet [Y. LeCun, et al., 1998]

Convolutional Neural Network

139

□ AlexNet



<https://learnopencv.com/understanding-alexnet/>

Convolutional Neural Network

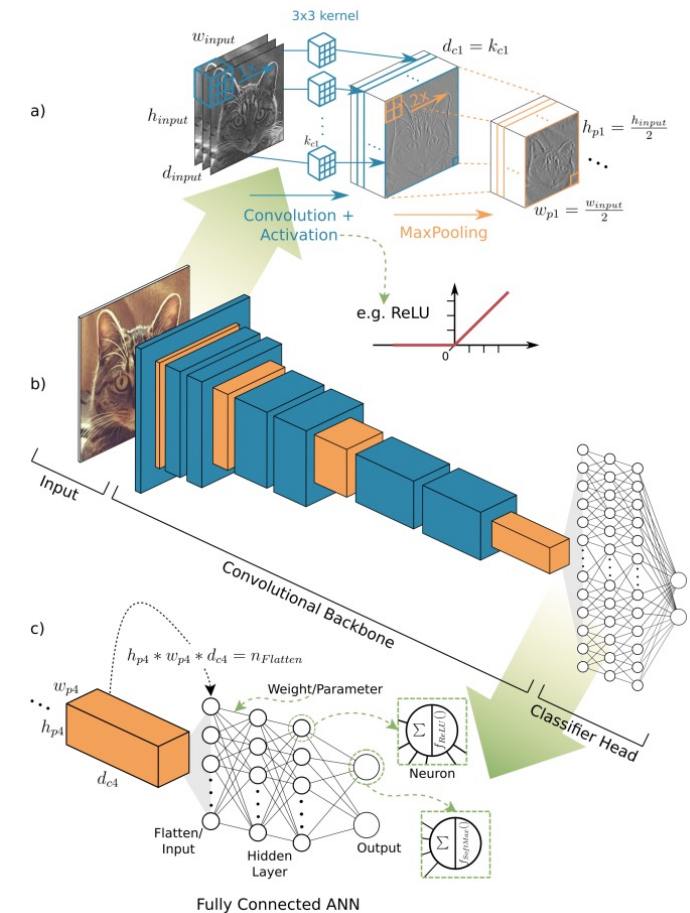
140

□ Backbone

- Convolutional Layer
 - ◆ Compute feature map
 - Filter
 - Randomly initialized
 - Optimize parameter by back-propagation
- Pooling Layer
 - ◆ Subsampling feature map
 - Max Pooling
 - Average Pooling

□ Head

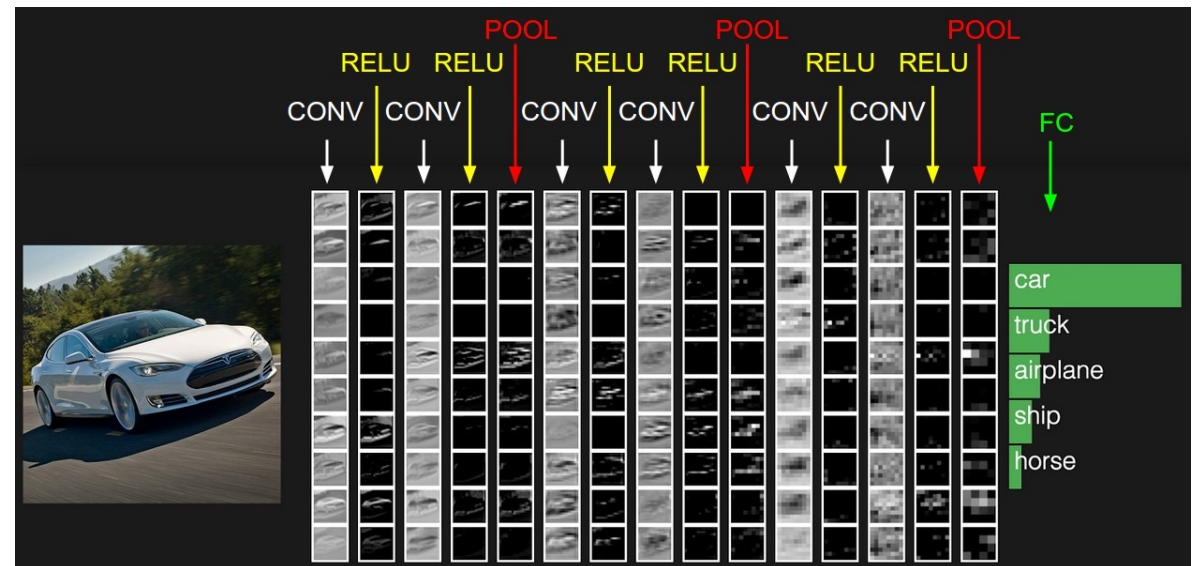
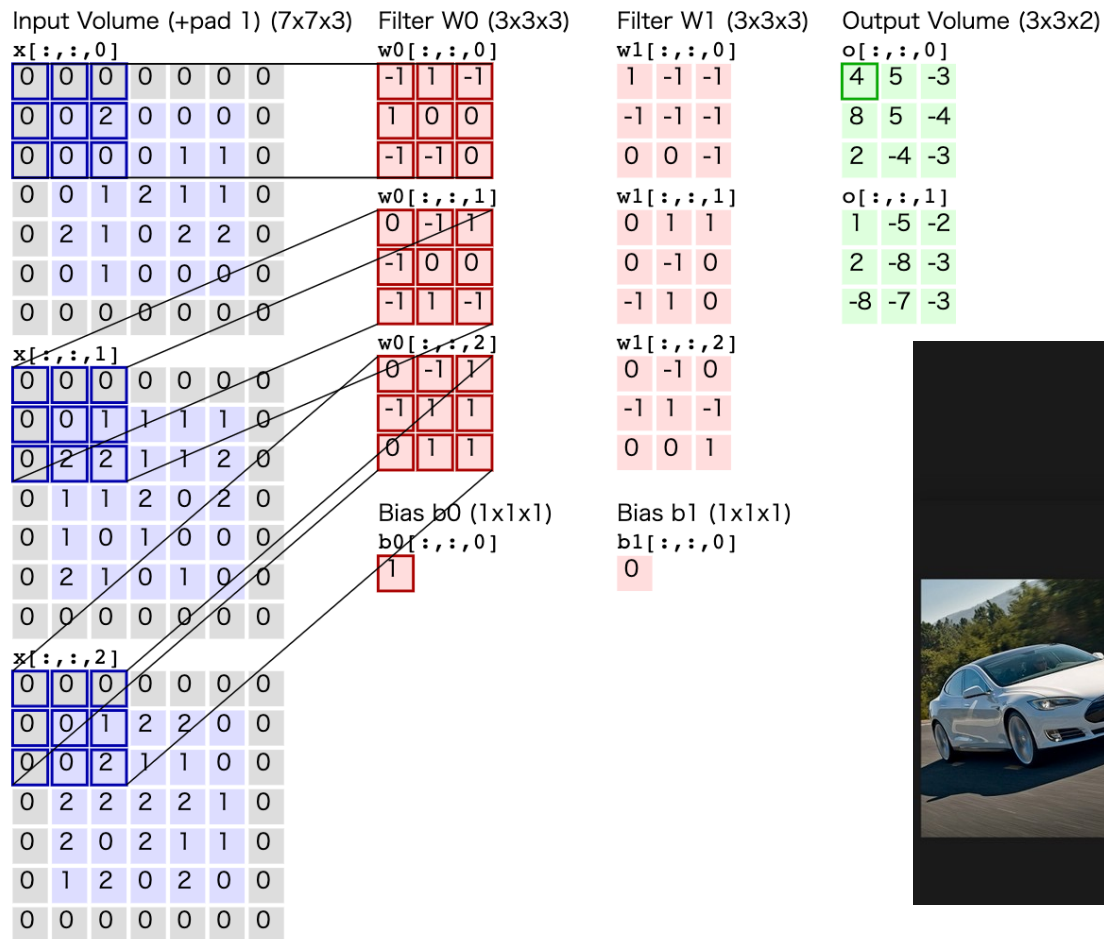
- Full Connection Layer
 - ◆ Feature Map is converted to array data (flatten)
 - Connect to MLP



[T. Hoeser, C. Kuenzer, Object detection and Image segmentation with Deep learning on earth observation data: a review part I: Evolution and Recent Trends, remote sensing, 2020]

Convolutional Neural Network

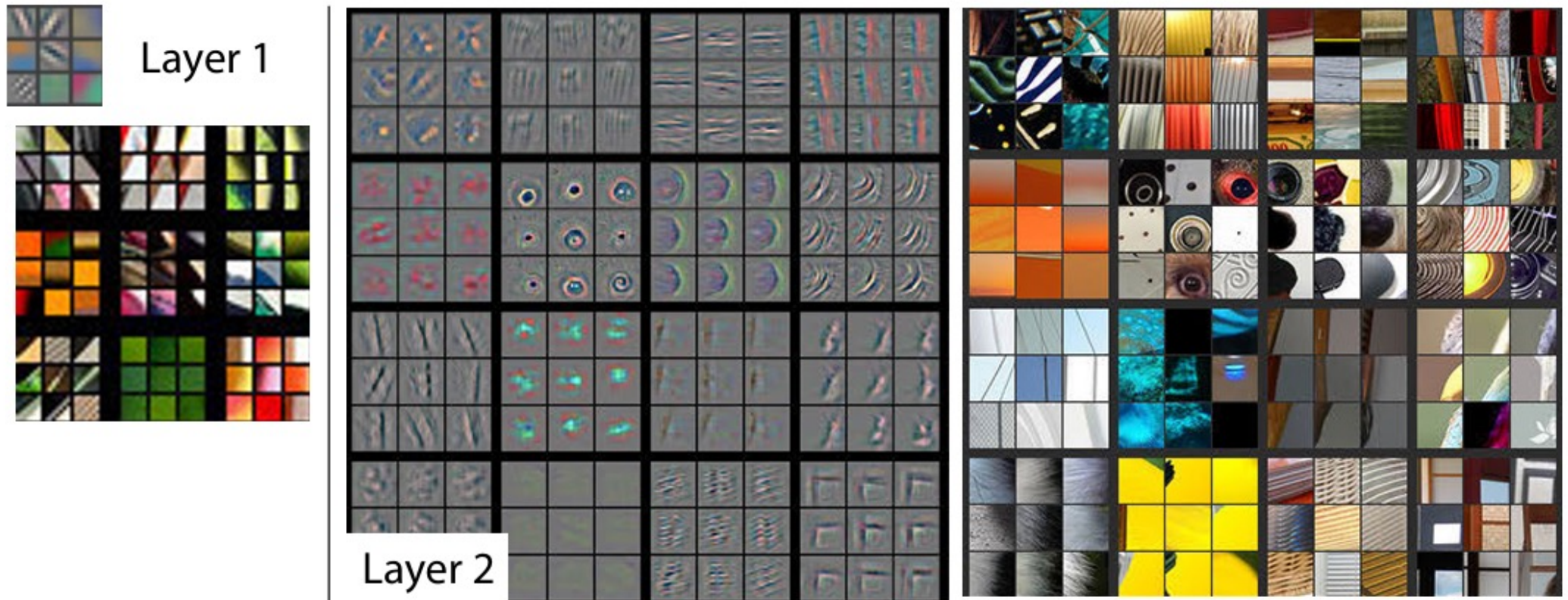
141



<https://cs231n.github.io/convolutional-networks/>

Visualizing CNN Feature

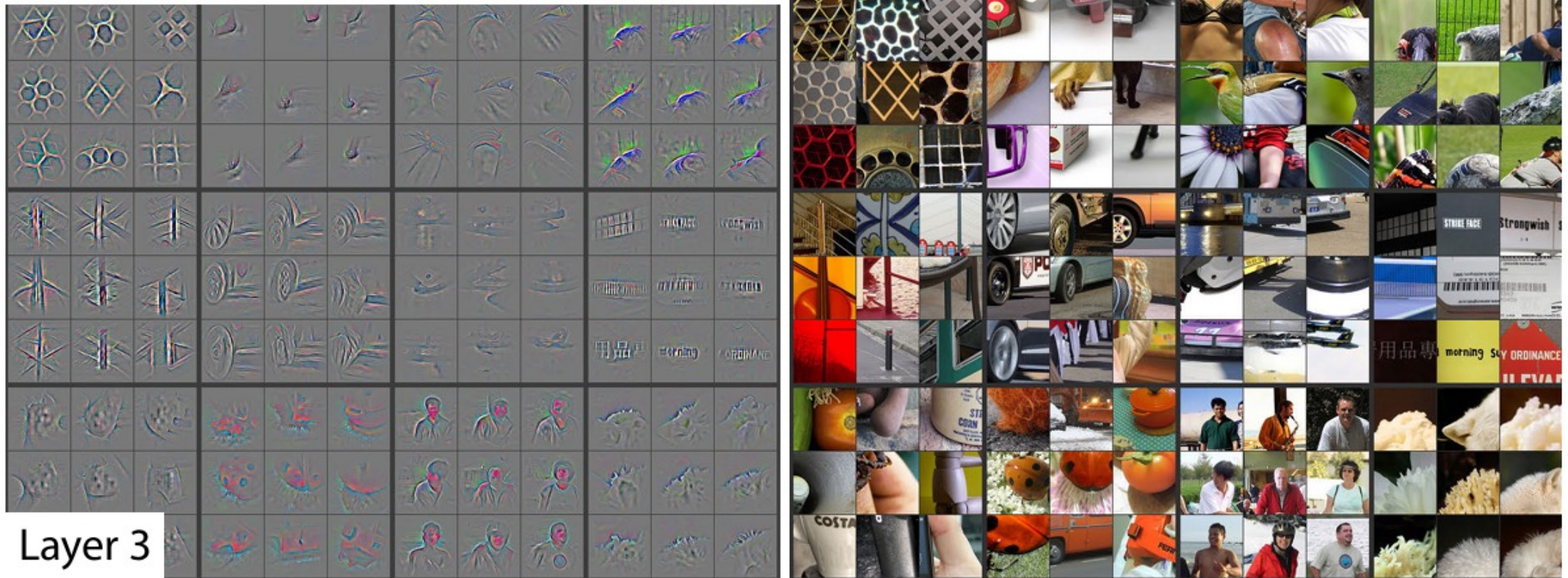
142



[M. D. Zeiler, R. Fergus, Visualizing and Understanding Convolutional Networks, 2013]

Visualizing CNN Feature

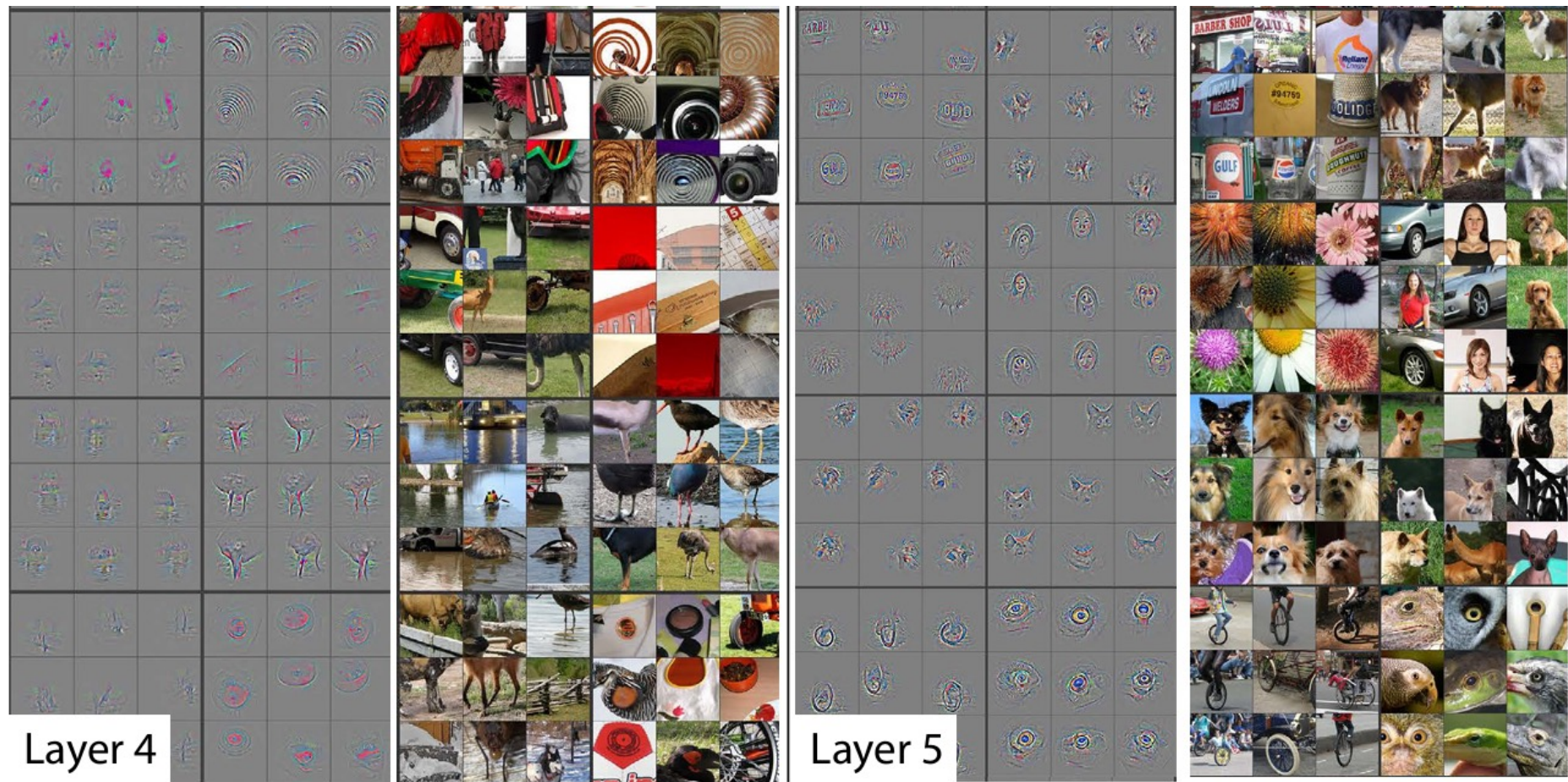
143



[M. D. Zeiler, R. Fergus, Visualizing and Understanding Convolutional Networks, 2013]

Visualizing CNN Feature

144



[M. D. Zeiler, R. Fergus, Visualizing and Understanding Convolutional Networks, 2013]

Example: MNIST

145

□ Train mapping function for provided dataset

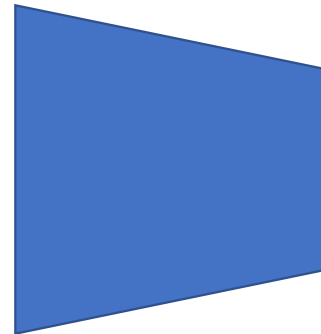
- $y = f(x)$
 - ◆ x : input vector (n-dim)
 - ◆ y : output vector (m-dim)
 - ◆ $f(\cdot)$: mapping function

0 0 0 0 0 0 0 0 0 0
1 1 1 1 1 1 1 1 1 1
2 2 2 2 2 2 2 2 2 2
3 3 3 3 3 3 3 3 3 3
4 4 4 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5 5
6 6 6 6 6 6 6 6 6 6
7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9

MNIST Dataset



input



mapping function
(algorithm)

task 0?, 1?, 2?, 3?, ...
score [0.0, 0.05, 0.2, 0.7, ...]

task even?, odd?
score [0.3, 0.7]

task 3?, or not?
score [0.9, 0.1]
output

Example: MNIST Classification

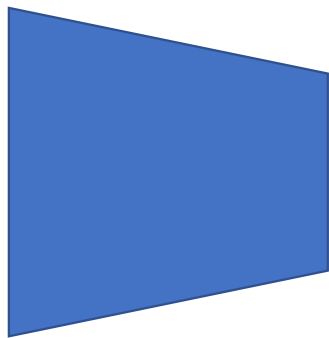
146



Input (28x28px)



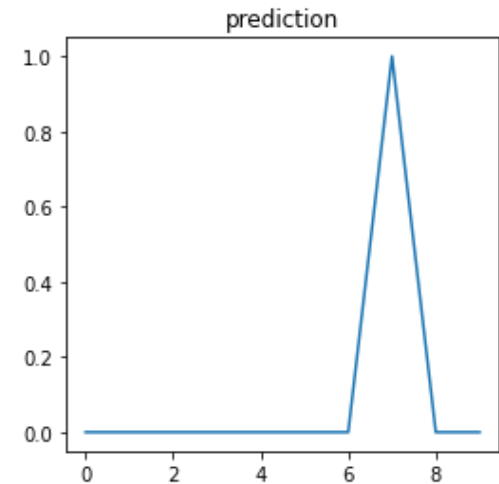
Input (784dim)



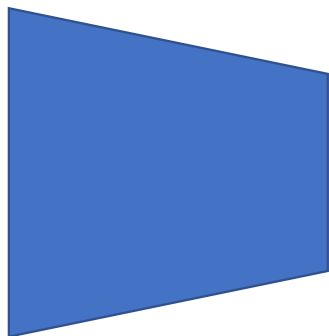
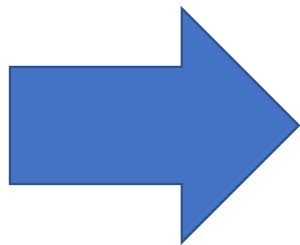
MLP



Output (10dim)



Input (28x28px)



CNN



Output (10dim)

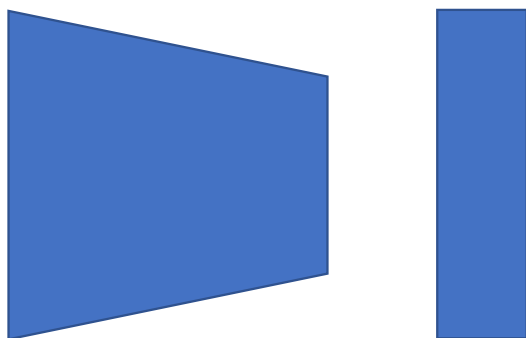
task 0?, 1?, 2?, 3?, ...
score [0.0, 0.05, 0.2, 0.7, ...]

MNIST: Cross Entropy (1/4)

147



Input (28x28px)



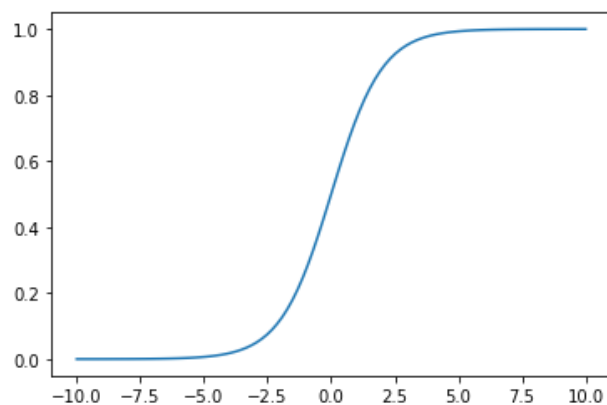
Output (10dim)

Label: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0]

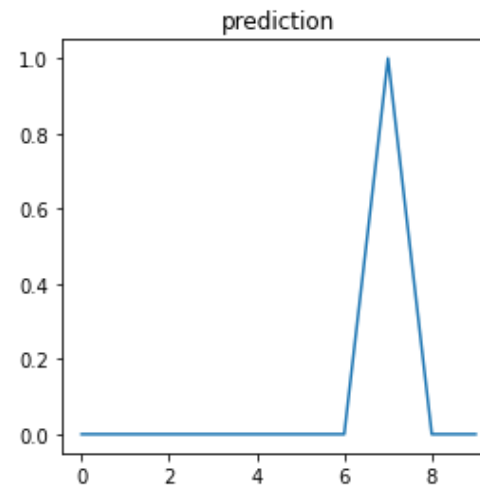


How to compute error??

Output: [0.1, 0.3, 0.2, 0.1, 0.1, 0.5, 0.1, 5.0, 0.2, 0.1]



sigmoid

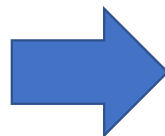
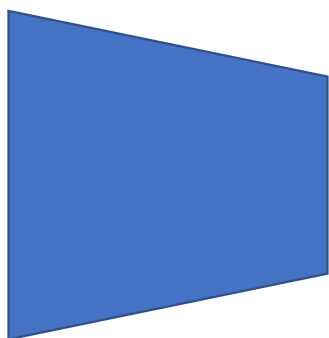


MNIST: Cross Entropy (2/4)

148



Input (28x28px)



Output o_i

Softmax

Cross-Entropy Loss

$$E = - \sum_{i=1}^N \hat{y}_i \log y_i$$

$$y_i = \frac{e^{o_i}}{\sum e^{o_j}} \quad \sum y_i = 1$$

$$\frac{\partial E}{\partial o_i} = \sum_k \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial o_i}$$

$$\frac{\partial E}{\partial y_k} = -\frac{\hat{y}_k}{y_k} \quad \because (\log x)' = \frac{1}{x}$$

$$(if \ k \neq i) \quad \frac{\partial y_k}{\partial o_i} = \frac{-e^{o_k} e^{o_i}}{(\sum e^{o_j})^2} = -\frac{e^{o_k}}{\sum e^{o_j}} \frac{e^{o_i}}{\sum e^{o_j}} = -y_k y_i \quad \because \left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$$

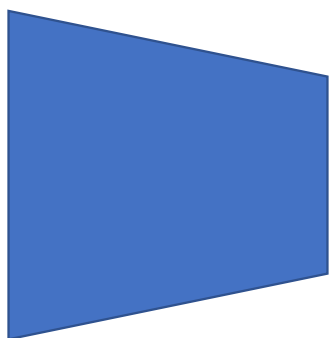
$$(if \ k = i) \quad \frac{\partial y_i}{\partial o_i} = \frac{e^{o_i} \sum e^{o_j} - e^{o_i} e^{o_i}}{(\sum e^{o_j})^2} = \frac{e^{o_i} (\sum e^{o_j} - e^{o_i})}{(\sum e^{o_j})^2} \frac{e^{o_i}}{\sum e^{o_j}} \frac{(\sum e^{o_j} - e^{o_i})}{\sum e^{o_j}} = y_i (1 - y_i)$$

MNIST: Cross Entropy (3/4)

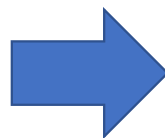
149



Input (28x28px)



Output o_i



Softmax

Cross-Entropy Loss

$$E = - \sum_{i=1}^N \hat{y}_i \log y_i$$

$$y_i = \frac{e^{o_i}}{\sum e^{o_j}} \quad \sum y_i = 1$$

$$\frac{\partial E}{\partial o_i} = \sum_k \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial o_i}$$

$$\frac{\partial E}{\partial y_k} = -\frac{\hat{y}_k}{y_k}$$

$$(if \ k \neq i) \quad \frac{\partial y_k}{\partial o_i} = -y_k y_i$$

$$(if \ k = i) \quad \frac{\partial y_i}{\partial o_i} = y_i(1 - y_i)$$

$$\frac{\partial E}{\partial o_i} = \sum_k \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial o_i} = \frac{\partial E}{\partial y_i} \frac{\partial y_i}{\partial o_i} + \sum_{k \neq i} \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial o_i}$$

$$= -\frac{\hat{y}_i}{y_i} y_i(1 - y_i) + \sum_{k \neq i} \left(-\frac{\hat{y}_k}{y_k}\right) (-y_k y_i)$$

$$= -\hat{y}_i(1 - y_i) + \sum_{k \neq i} \hat{y}_k y_i = -\hat{y}_i + \sum_k \hat{y}_k y_i = -\hat{y}_i + y_i \sum_k \hat{y}_k$$

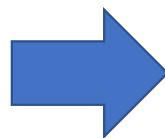
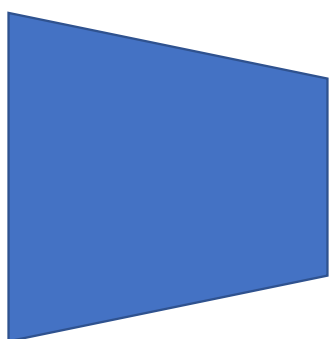
$$\frac{\partial E}{\partial o_i} = y_i - \hat{y}_i$$

MNIST: Cross Entropy (4/4)

150

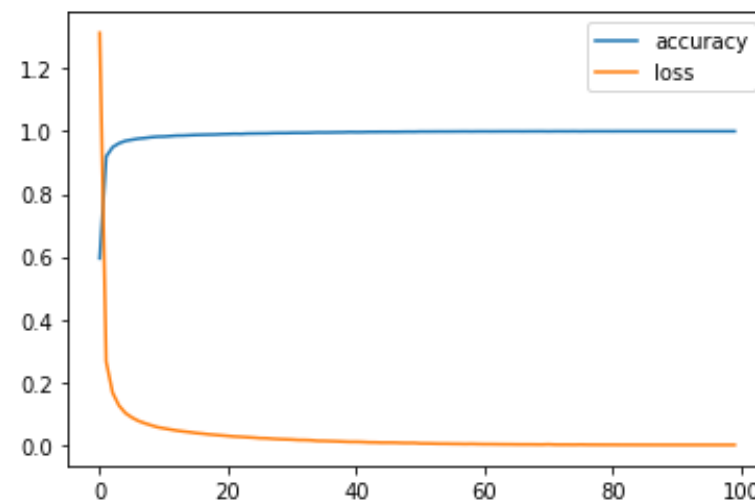
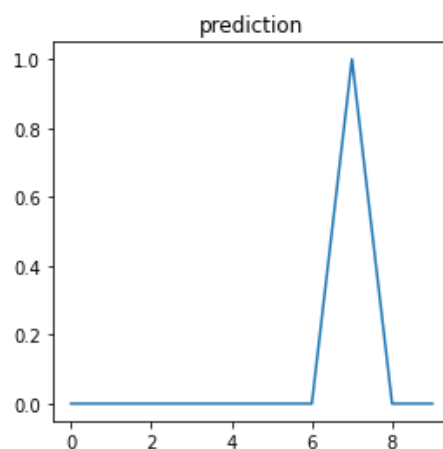
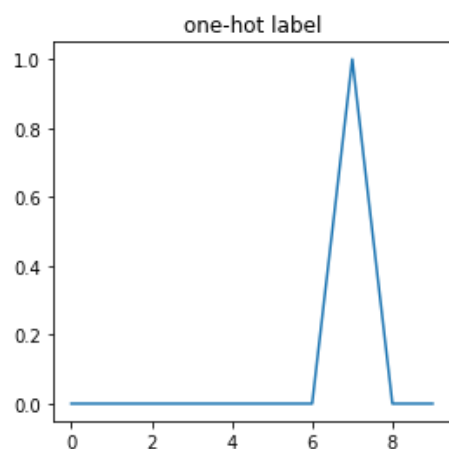


Input (28x28px)



Output o_i

Softmax

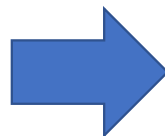
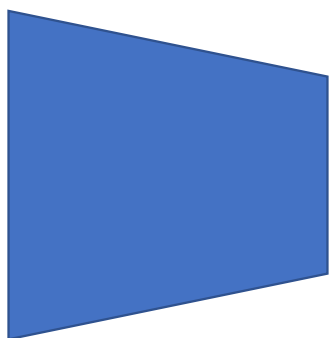


MNIST: Binary Cross Entropy (1/3)

151



Input (28x28px)



Output (1dim)

o

Sigmoid (1dim)

$$y = \frac{1}{1 + e^{-o}}$$

Label: [1.0]

Even Number: 0

Odd Number: 1



Output: [0.8]

Cross-Entropy Loss

$$E = - \sum_{i=1}^N \hat{y}_i \log y_i$$

Binary Cross-Entropy Loss

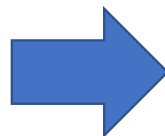
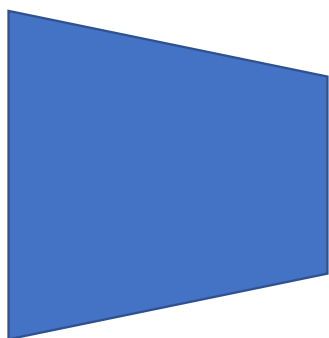
$$E = -\hat{y} \log y - (1 - \hat{y}) \log(1 - y)$$

MNIST: Binary Cross Entropy (2/3)

152



Input (28x28px)



Output (1dim)

o

Sigmoid (1dim)

$$y = \frac{1}{1 + e^{-o}}$$

Label: [1.0]

Even Number: 0

Odd Number: 1



Output: [0.8]

Binary Cross-Entropy Loss

$$E = -\hat{y} \log y - (1 - \hat{y}) \log(1 - y)$$

$$\frac{\partial E}{\partial o} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial o}$$

$$\frac{\partial E}{\partial y} = -\frac{\hat{y}}{y} - (1 - \hat{y}) \frac{-1}{1 - y} = -\frac{\hat{y}}{y} + \frac{1 - \hat{y}}{1 - y}$$

$$= -\frac{\hat{y}(1 - y)}{y(1 - y)} + \frac{y(1 - \hat{y})}{y(1 - y)} = \frac{-\hat{y} + y\hat{y} + y - y\hat{y}}{y(1 - y)} = \frac{y - \hat{y}}{y(1 - y)}$$

$$\frac{\partial y}{\partial o} = \frac{\partial}{\partial o} (1 + e^{-o})^{-1} = (-1)(1 + e^{-o})^{-2} e^{-o} (-1) = \frac{e^{-o}}{(1 + e^{-o})^2} = \frac{1 - 1 + e^{-o}}{1 + e^{-o}} \frac{1}{1 + e^{-o}}$$

$$= \left(\frac{1 + e^{-o} - 1}{1 + e^{-o}} \right) y = \left(1 - \frac{1}{1 + e^{-o}} \right) y = (1 - y)y$$

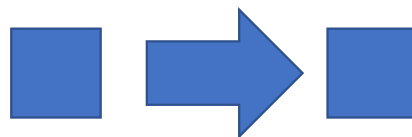
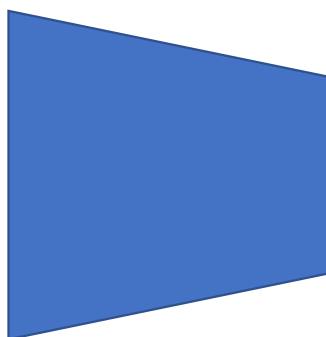
$$\therefore \frac{\partial E}{\partial o} = \frac{y - \hat{y}}{y(1 - y)} (1 - y)y = y - \hat{y}$$

MNIST: Binary Cross Entropy (3/3)

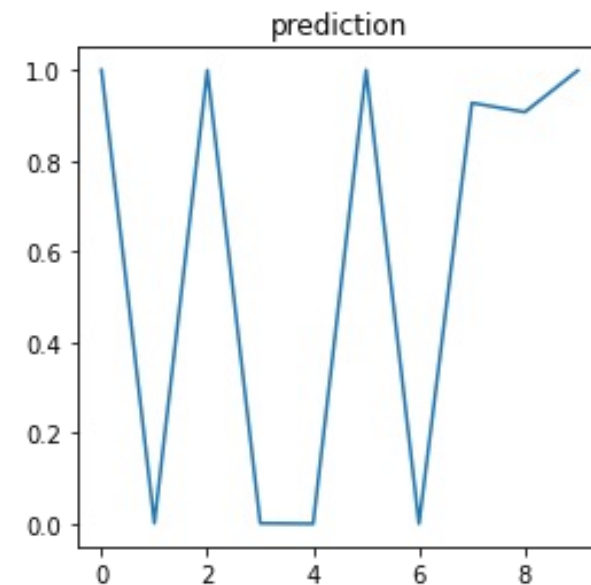
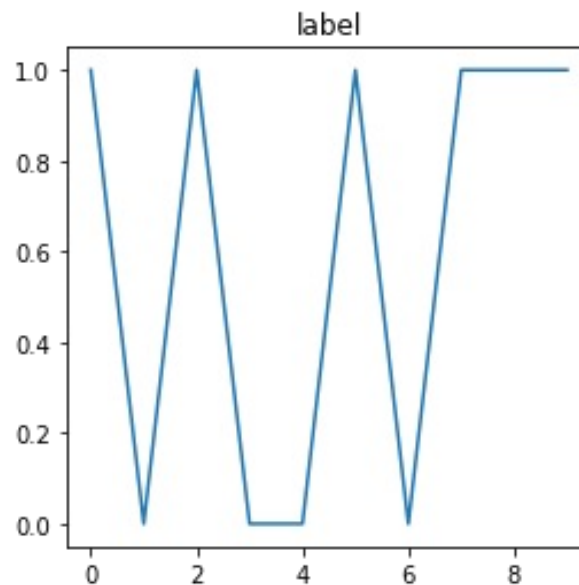
153



Input (28x28px)



Output (1dim) Sigmoid (1dim)



Application: Image recognition

154

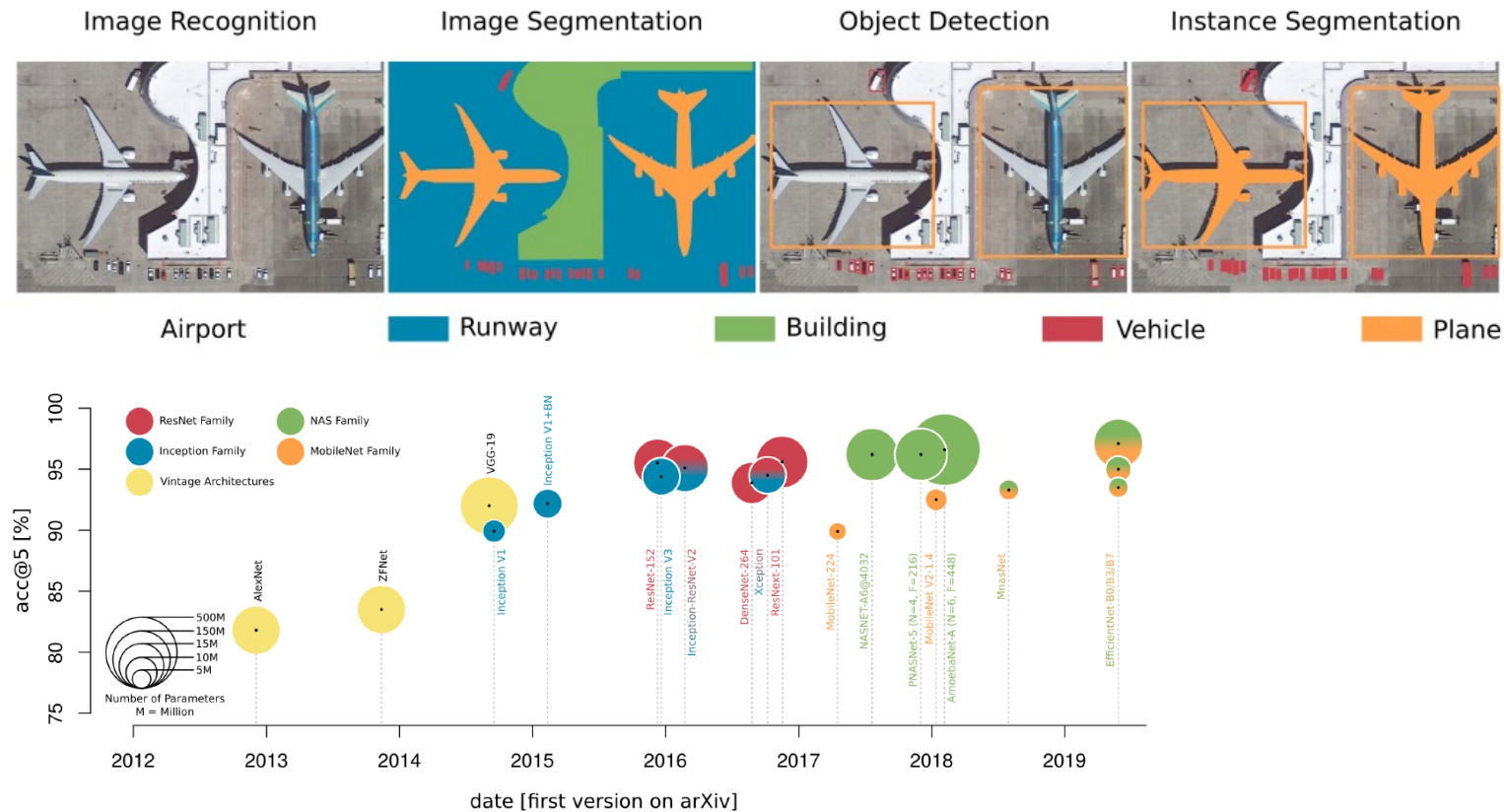


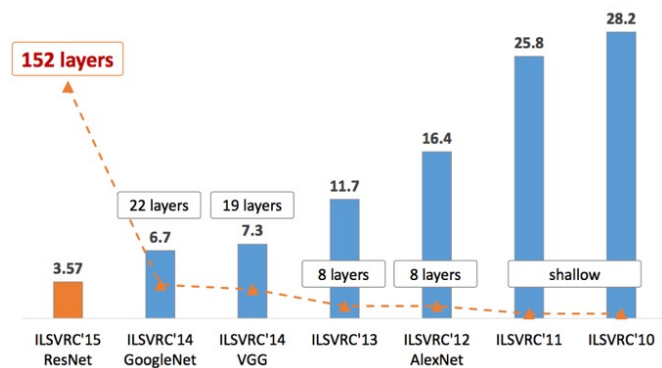
Figure 4. Evolution of milestone architectures for image recognition from 2012 to 2019, compared by their accuracy ($acc@5$) metric on the ImageNet 2012 dataset and showing their numbers of parameters in log scale as size of the circle. The different colours show relations between the architectures; mixed colours mean that concepts of two architecture families are combined in those architectures.

[T. Hoeser, C. Kuenzer, Object detection and Image segmentation with Deep learning on earth observation data: a review part I: Evolution and Recent Trends, remote sensing, 2020]

Application: Classification (1/2)

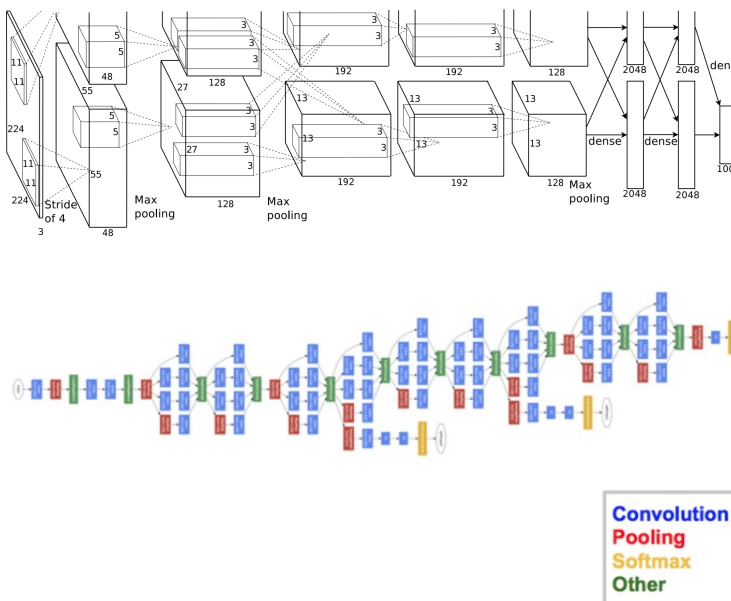
155

- Input: Image
- Output: Category

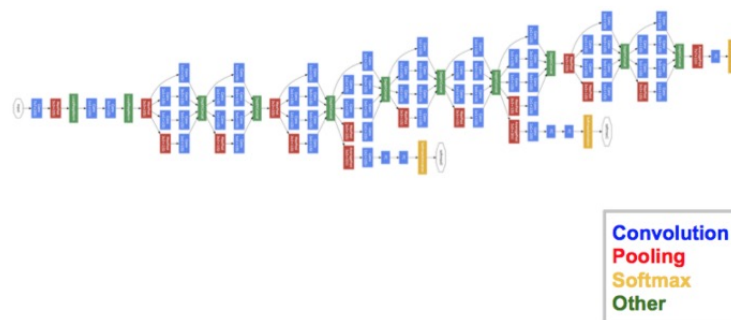


Error rate for ILSVRC
(ImageNet Large Scale Visual
Recognition Challenge)

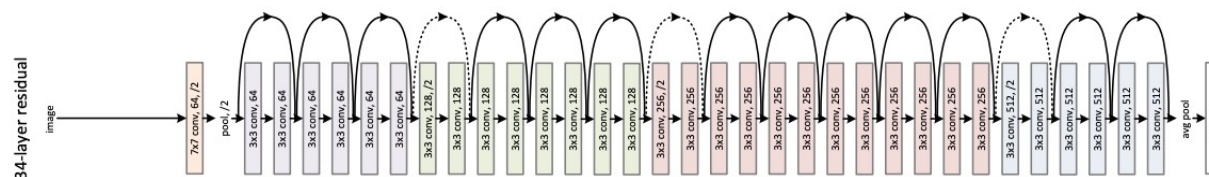
Classification Task
Number of Images: 14M
Number of Classes: More than 20,000



AlexNet
[A. Krizhevsky, et al., 2012]



GoogLeNet
[C. Szegedy, et al., 2014]



ResNet [K. He, et al., 2015]

Application: Classification (2/2)

156

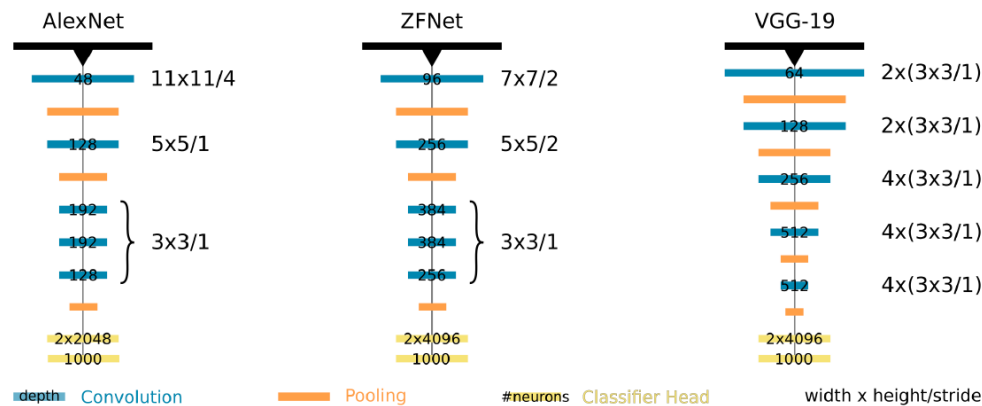


Figure 5. Conceptual overview of the *Vintage Architectures*. The architecture designs of AlexNet [3] and ZFNet [38] are similar despite smaller kernel in the first convolutional layers and deeper feature maps in ZFNet, whereas VGG-19 [39] is considerably deeper overall and it uses a uniform kernel size.

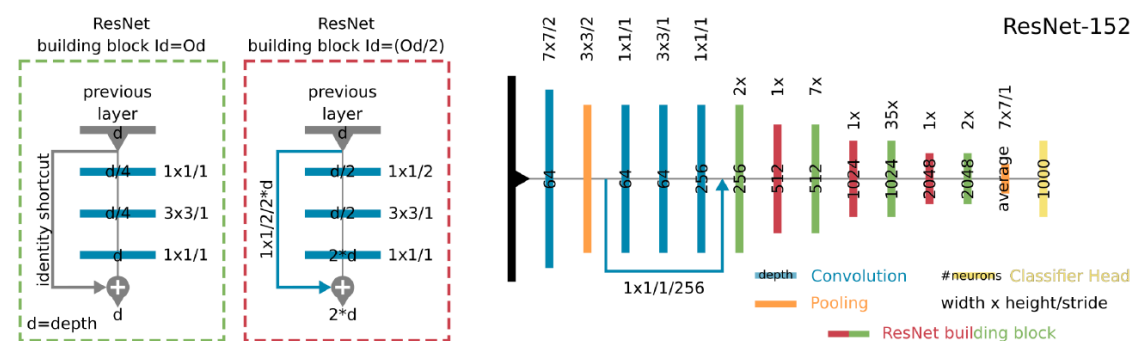


Figure 7. Conceptual overview of the ResNet building block and the ResNet-152 architecture [43]: (Left) The two ResNet building blocks show the convolutional operations in the main trunk, extracting features and the residual connections surrounding these operations to enable a more efficient transport of gradients during training. The difference between the two variants is that the first transports the input value of the block without changing it (identity shortcut) where the second uses convolution with stride 2, to match the output resolution and depth of the main trunk. (Right) Overall architecture of ResNet-152 consists of 152 layers due to stacked ResNet building blocks.

[T. Hoeser, C. Kuenzer, Object detection and Image segmentation with Deep learning on earth observation data: a review part I: Evolution and Recent Trends, remote sensing, 2020]

Application: Regression

157

- Input: Image
- Output: Vector

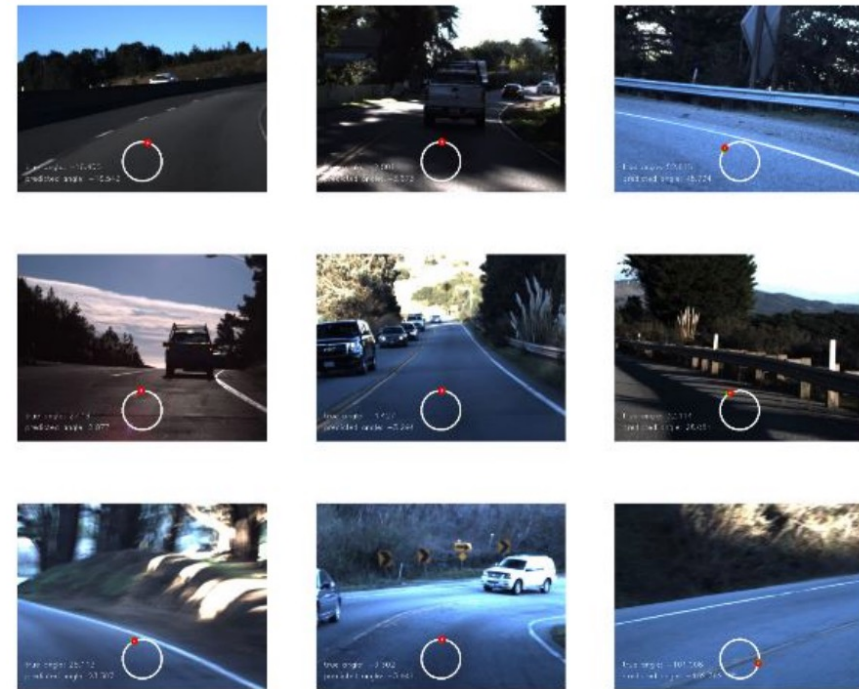
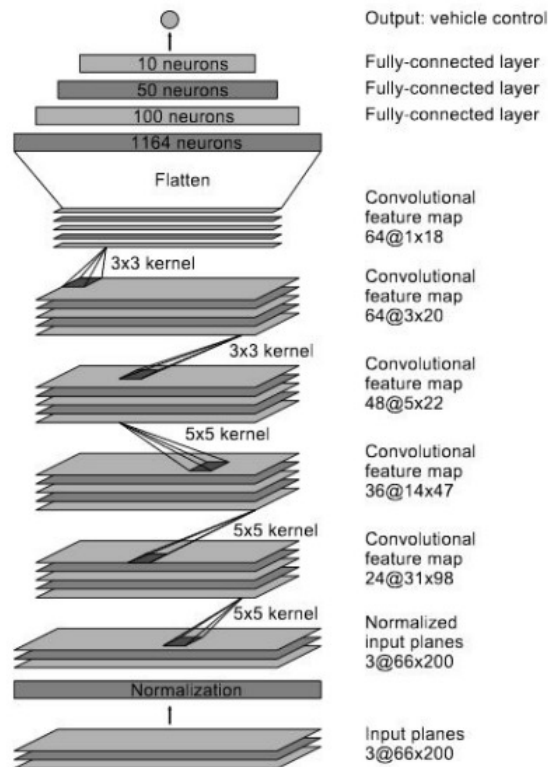


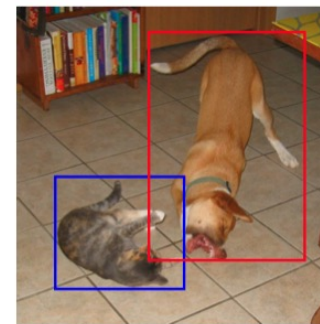
Figure 13. Example actual vs. predicted angle on unprocessed images (transfer model).

[S. Do, et al., 2017]

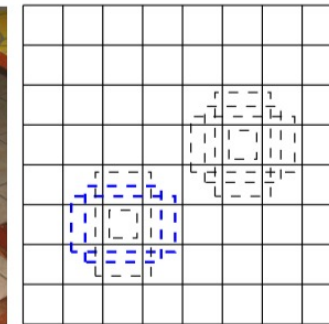
Application: Detection

158

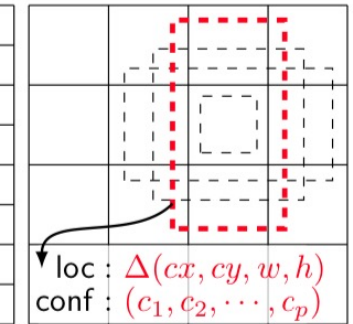
- Input: Image
- Output: Bounding Box
 - Compute confidence value for default boxes



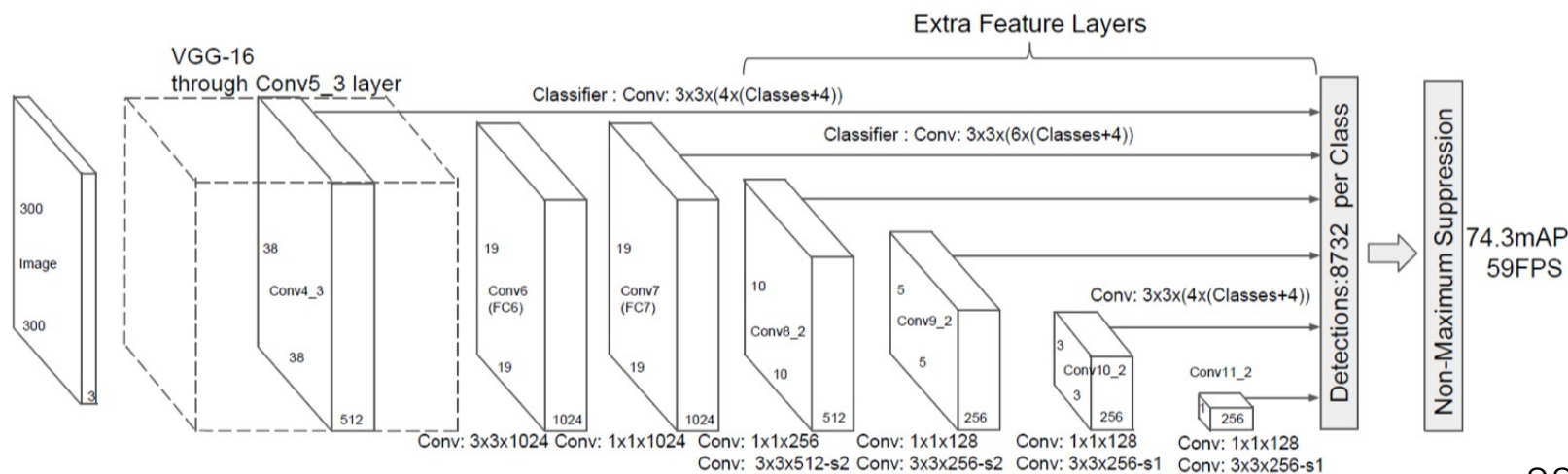
(a) Image with GT boxes



(b) 8×8 feature map



(c) 4×4 feature map



SSD [W. Liu, et al., 2016]

Application: Segmentation

159

- Input: Image
- Output: Image (Segmentation Result)

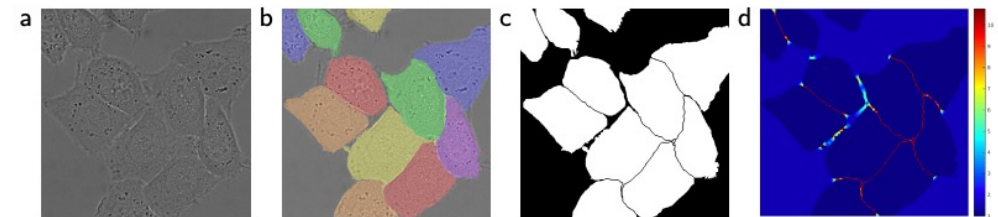
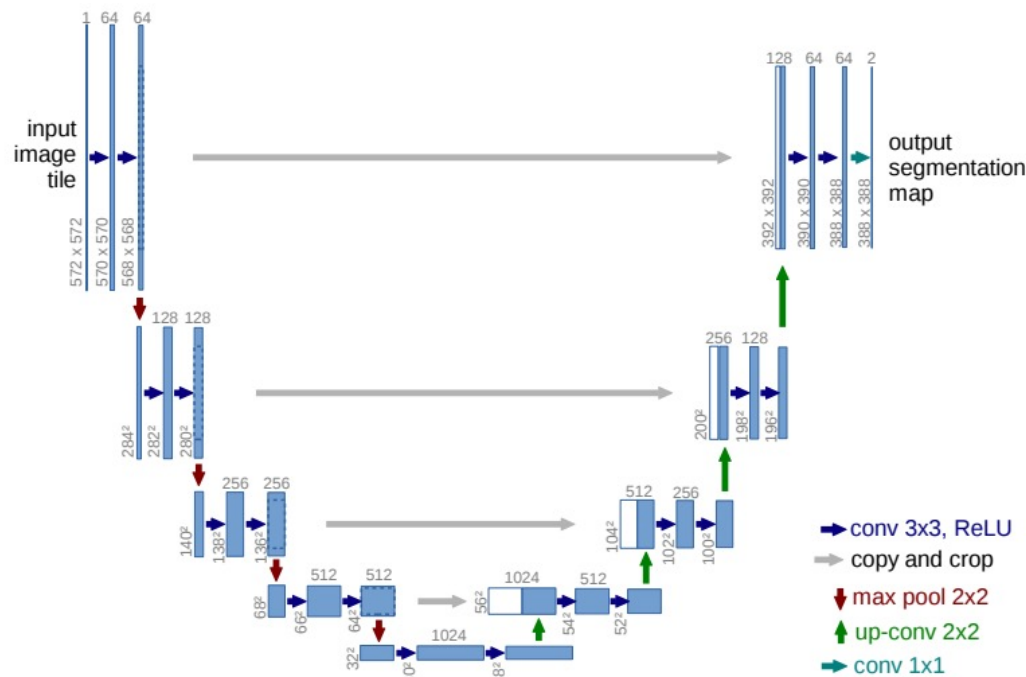


Fig. 3. HeLa cells on glass recorded with DIC (differential interference contrast) microscopy. (a) raw image. (b) overlay with ground truth segmentation. Different colors indicate different instances of the HeLa cells. (c) generated segmentation mask (white: foreground, black: background). (d) map with a pixel-wise loss weight to force the network to learn the border pixels.

U-Net [O. Ronneberger, et al., 2015]

Neural Network for Point Cloud

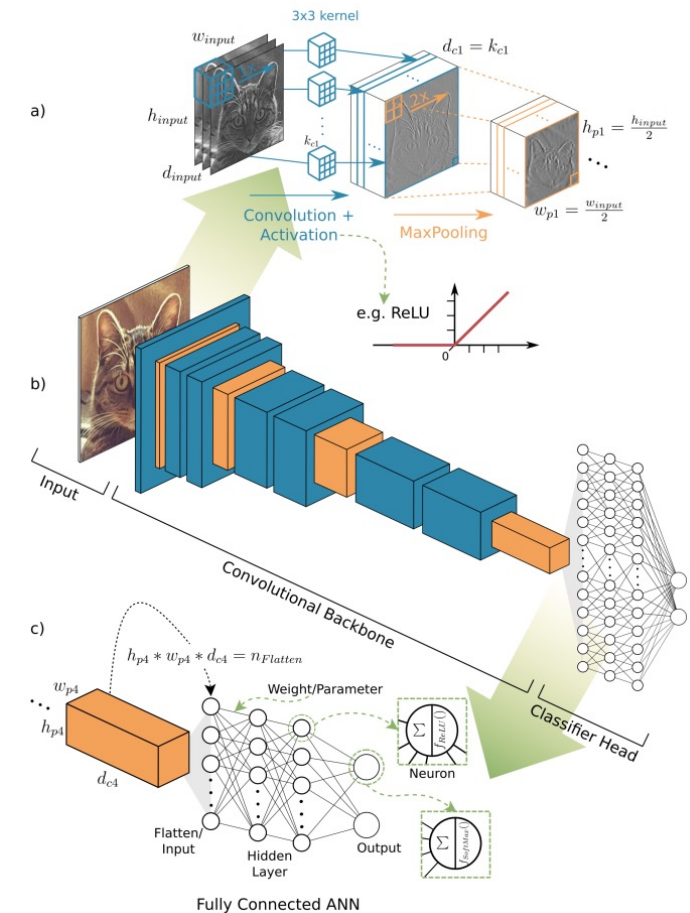
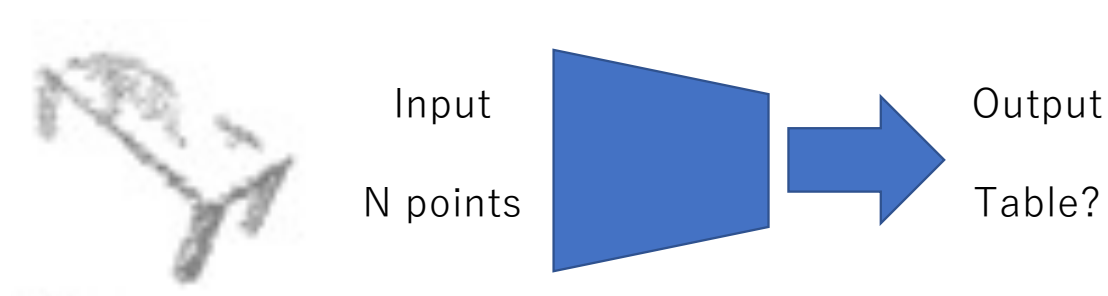
160

□ Neural Network for Image input

- Extract local features in 2D space
 - ◆ It is important to maintain and enter order information for each pixel.

□ Neural Network for Point Cloud input

- There is no meaning for the point cloud order
 - ◆ Two approaches of feature extraction for point cloud
 - Input data in random order and extract features
 - Input set data and extract features



Neural Network for Point Cloud

161

Input set data and extract features

- Input order and transformation Invariant

- Unordered feature extraction

- $f(x_1, x_2, x_3, \dots) = f(x_3, x_1, x_2, \dots)$

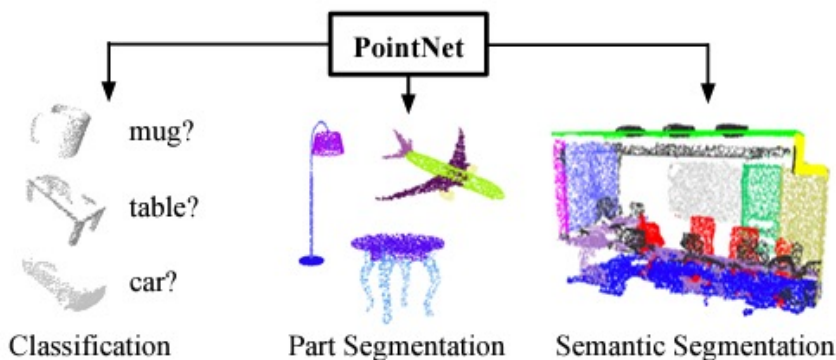
For example...

$$f(x_1, x_2, x_3, \dots) = \sum x_i$$
$$f(x_1, x_2, x_3, \dots) = \max(x_1, x_2, x_3, \dots)$$

- Invariant under transformation (shift, rotation)

- $f(x_1, x_2, x_3, \dots) = f(x_1 + r, x_2 + r, x_3 + r, \dots), f(x_1, x_2, x_3, \dots) = f(Rx_1, Rx_2, Rx_3, \dots)$

PointNet



[C. R. Qi, H. Su, K. Mo, L. J. Guibas, PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation, 2016]

Multiply matrix for all points, apply maxpool for each channel to extract global and local feature

